

Timing Verification

This chapter describes the checks that are performed as part of static timing analysis. These checks are intended to exhaustively verify the timing of the design under analysis.

The two primary checks are the setup and hold checks. Once a clock is defined at the clock pin of a flip-flop, setup and hold checks are automatically inferred for the flip-flop. The timing checks are generally performed at multiple conditions including the worst-case slow condition and best-case fast condition. Typically, the worst-case slow condition is critical for setup checks and best-case fast condition is critical for hold checks - though the hold checks may be performed at the worst-case slow condition also.

The examples presented in this chapter assume that the net delays are zero; this is done for simplicity and does not alter the concepts presented herein.

8.1 Setup Timing Check

A **setup timing check** verifies the timing relationship between the clock and the data pin of a flip-flop so that the setup requirement is met. In other words, the setup check ensures that the data is available at the input of the flip-flop before it is clocked in the flip-flop. The data should be stable for a certain amount of time, namely the setup time of the flip-flop, before the active edge of the clock arrives at the flip-flop. This requirement ensures that the data is captured reliably into the flip-flop. Figure 8-1 shows the setup requirement of a typical flip-flop. A setup check verifies the setup requirement of the flip-flop.

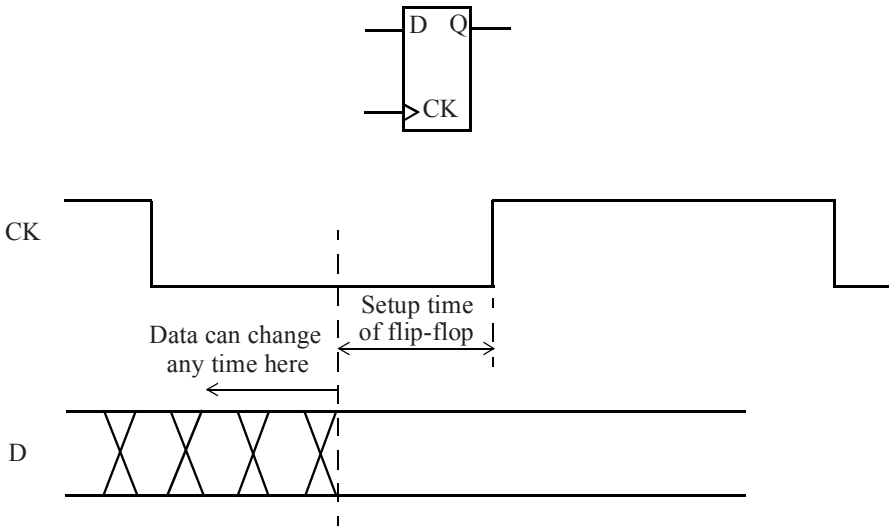


Figure 8-1 Setup requirement of a flip-flop.

In general, there is a launch flip-flop - the flip-flop that launches the data, and a capture flip-flop - the flip-flop that captures the data whose setup time must be satisfied. The setup check validates the long (or max) path from the launch flip-flop to the capture flip-flop. The clocks to these two flip-flops can be the same or can be different. The setup check is from the first active edge of the clock in the launch flip-flop to the closest following

active edge of the capture flip-flop. The setup check ensures that the data launched from the previous clock cycle is ready to be captured after one cycle.

We now examine a simple example, shown in Figure 8-2, where both the launch and capture flip-flops have the same clock. The first rising edge of clock $CLKM$ appears at time T_{launch} at launch flip-flop. The data launched by this clock edge appears at time $T_{launch} + T_{ck2q} + T_{dp}$ at the D pin of the flip-flop $UFF1$. The second rising edge of the clock (setup is normally checked after one cycle) appears at time $T_{cycle} + T_{capture}$ at the clock pin of the capture flip-flop $UFF1$. The difference between these two times must be larger than the setup time of the flip-flop, so that the data can be reliably captured in the flip-flop.

The setup check can be mathematically expressed as:

$$T_{launch} + T_{ck2q} + T_{dp} < T_{capture} + T_{cycle} - T_{setup}$$

where T_{launch} is the delay of the clock tree of the launch flip-flop $UFF0$, T_{dp} is the delay of the combinational logic data path and T_{cycle} is the clock period. $T_{capture}$ is the delay of the clock tree for the capture flip-flop $UFF1$.

In other words, the total time it takes for data to arrive at the D pin of the capture flip-flop must be less than the time it takes for the clock to travel to the capture flip-flop plus a clock cycle delay minus the setup time.

Since the setup check poses a $\max^1$ constraint, the setup check *always* uses the longest or the max timing path. For the same reason, this check is normally verified at the slow corner where the delays are the largest.

1. It imposes an *upper bound* on the data path delay.

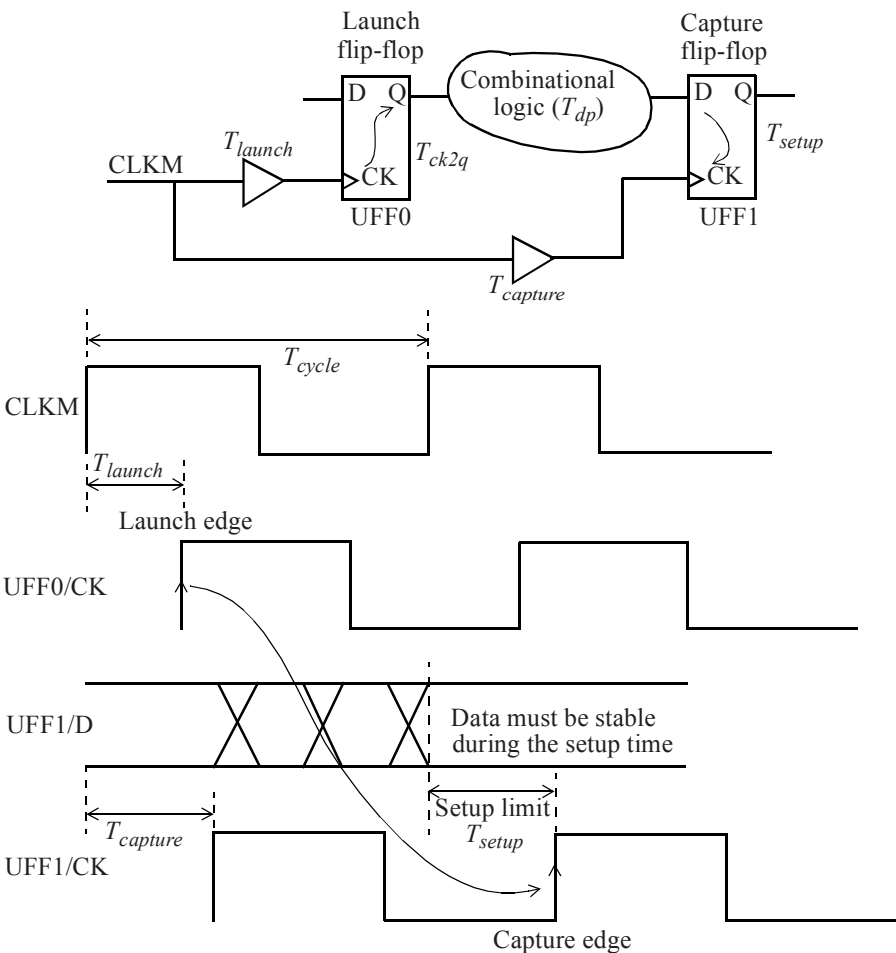


Figure 8-2 Data and clock signals for setup timing check.

8.1.1 Flip-flop to Flip-flop Path

Here is a path report of a setup check.

Startpoint: UFF0 (rising edge-triggered flip-flop clocked by CLKM)
Endpoint: UFF1 (rising edge-triggered flip-flop clocked by CLKM)
Path Group: CLKM
Path Type: max

Point	Incr	Path

clock CLKM (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
UFF0/CK (DFF)	0.00	0.00 r
UFF0/Q (DFF) <-	0.16	0.16 f
UNOR0/ZN (NR2)	0.04	0.20 r
UBUF4/Z (BUFF)	0.05	0.26 r
UFF1/D (DFF)	0.00	0.26 r
data arrival time		0.26
clock CLKM (rise edge)	10.00	10.00
clock network delay (ideal)	0.00	10.00
clock uncertainty	-0.30	9.70
UFF1/CK (DFF)		9.70 r
library setup time	-0.04	9.66
data required time		9.66

data required time		9.66
data arrival time		-0.26

slack (MET)		9.41

The report shows that the launch flip-flop (specified by *Startpoint*) has instance name *UFF0* and it is triggered by the rising edge of clock *CLKM*. The capture flip-flop (specified by *Endpoint*) is *UFF1* and is also triggered by the rising edge of clock *CLKM*. The *Path Group* line indicates that it belongs to the path group *CLKM*. As discussed in the previous chapter, all paths in a design are categorized into path groups based on the clock of the capture flip-flop. The *Path Type* line indicates that the delays shown in this report

are all max path delays indicating that this is a setup check. This is because setup checks correspond to the max (or longest path) delays through the logic. Note that the hold checks correspond to the min (or shortest path) delays through the logic.

The *Incr* column specifies the incremental cell or net delay for the port or pin indicated. The *Path* column shows the cumulative delay for the arrival and the data required paths. Here is the clock specification used for this example.

```
create_clock -name CLKM -period 10 -waveform {0 5} \  
  [get_ports CLKM]  
set_clock_uncertainty -setup 0.3 [all_clocks]  
set_clock_transition -rise 0.2 [all_clocks]  
set_clock_transition -fall 0.15 [all_clocks]
```

The launch path takes 0.26ns to get to the *D* pin of flip-flop *UFF1* - this is the arrival time at the input of the capture flip-flop. The capture edge (which is one cycle away since this is a setup check) is at 10ns. A clock uncertainty of 0.3ns was specified for this clock - thus, the clock period is reduced by the uncertainty margin. The clock uncertainty includes the variation in cycle time due to jitter in the clock source and any other timing margin used for analysis. The setup time of the flip-flop 0.04ns (called *library setup time*), is deducted from the total capture path yielding a required time of 9.66ns. Since the arrival time is 0.26ns, there is a positive slack of 9.41ns on this timing path. Note that the difference between the required time and arrival time may appear to be 9.40ns - however the actual value is 9.41ns which appears in the report. The anomaly exists because the report shows only two digits after the decimal whereas the internally computed and stored values have greater precision than those reported.

What is the *clock network delay* in the timing report and why is it marked as *ideal*? This line in the timing report indicates that the clock trees are treated as *ideal*, that any buffers in the clock path are assumed to have zero delay. Once the clock trees are built, the clock network can be marked as *propagated* - which causes the clock paths to show up with real delays, as shown in the next example timing report. The 0.11ns delay is the clock network de-

lay on the launch clock and the 0.12ns delay is the clock network delay on the capture flip-flop.

Startpoint: UFF0 (rising edge-triggered flip-flop clocked by CLKM)
 Endpoint: UFF1 (rising edge-triggered flip-flop clocked by CLKM)
 Path Group: CLKM
 Path Type: max

Point	Incr	Path
-----	-----	-----
clock CLKM (rise edge)	0.00	0.00
clock network delay (propagated)	0.11	0.11
UFF0/CK (DFF)	0.00	0.11 r
UFF0/Q (DFF) <-	0.14	0.26 f
UNOR0/ZN (NR2)	0.04	0.30 r
UBUF4/Z (BUFF)	0.05	0.35 r
UFF1/D (DFF)	0.00	0.35 r
data arrival time		0.35
clock CLKM (rise edge)	10.00	10.00
clock network delay (propagated)	0.12	10.12
clock uncertainty	-0.30	9.82
UFF1/CK (DFF)		9.82 r
library setup time	-0.04	9.78
data required time		9.78
-----	-----	-----
data required time		9.78
data arrival time		-0.35
-----	-----	-----
slack (MET)		9.43

The timing path report can optionally include the expanded clock paths, that is, with the clock trees explicitly shown. Here is such an example.

Startpoint: UFF0 (rising edge-triggered flip-flop clocked by CLKM)
 Endpoint: UFF1 (rising edge-triggered flip-flop clocked by CLKM)
 Path Group: CLKM
 Path Type: max

Point	Incr	Path

clock CLKM (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKM (in)	0.00	0.00 r
UCKBUF0/C (CKB)	0.06	0.06 r
UCKBUF1/C (CKB)	0.06	0.11 r
UFF0/CK (DFF)	0.00	0.11 r
UFF0/Q (DFF) <-	0.14	0.26 f
UNOR0/ZN (NR2)	0.04	0.30 r
UBUF4/Z (BUFF)	0.05	0.35 r
UFF1/D (DFF)	0.00	0.35 r
data arrival time		0.35
clock CLKM (rise edge)	10.00	10.00
clock source latency	0.00	10.00
CLKM (in)	0.00	10.00 r
UCKBUF0/C (CKB)	0.06	10.06 r
UCKBUF2/C (CKB)	0.07	10.12 r
UFF1/CK (DFF)	0.00	10.12 r
clock uncertainty	-0.30	9.82
library setup time	-0.04	9.78
data required time		9.78

data required time		9.78
data arrival time		-0.35

slack (MET)		9.43

Notice that the clock buffers, *UCKBUF0*, *UCKBUF1* and *UCKBUF2* appear in the path report above and provide details of how the clock tree delays are computed.

How is the delay of the first clock cell *UCKBUF0* computed? As described in previous chapters, the cell delay is calculated based on the input transition time and the output capacitance of the cell. Thus, the question is what transition time is used at the input of the first cell in the clock tree. The transition time (or slew) on the input pin of the first clock cell can be explicitly specified using the *set_input_transition* command.


```
set_input_transition -rise 0.3 [get_ports CLKM]
set_input_transition -fall 0.45 [get_ports CLKM]
```

In the *set_input_transition* specification shown above, we specified the input rise transition time to be 0.3ns and the fall transition time to be 0.45ns. In the absence of the input transition specifications, ideal slew is assumed at the origin of the clock tree, which implies that both the rise and fall transition times are 0ns.

The “r” and “f” characters in the timing report indicate the rising (and falling) edge of the clock or data signal. The previous path report shows the path starting from the falling edge of *UFF0/Q* and ending on the rising edge of *UFF1/D*. Since *UFF1/D* can be either 0 or 1, there can be a path ending at the falling edge of *UFF1/D* as well. Here is such a path.

```
Startpoint: UFF0 (rising edge-triggered flip-flop clocked by CLKM)
Endpoint: UFF1 (rising edge-triggered flip-flop clocked by CLKM)
Path Group: CLKM
Path Type: max
```

Point	Incr	Path
-----	-----	-----
clock CLKM (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKM (in)	0.00	0.00 r
UCKBUF0/C (CKB)	0.06	0.06 r
UCKBUF1/C (CKB)	0.06	0.11 r
UFF0/CK (DFF)	0.00	0.11 r
UFF0/Q (DFF) <-	0.14	0.26 r
UNOR0/ZN (NR2)	0.02	0.28 f
UBUF4/Z (BUFF)	0.06	0.33 f
UFF1/D (DFF)	0.00	0.33 f
data arrival time		0.33
clock CLKM (rise edge)	10.00	10.00
clock source latency	0.00	10.00
CLKM (in)	0.00	10.00 r
UCKBUF0/C (CKB)	0.06	10.06 r
UCKBUF2/C (CKB)	0.07	10.12 r
UFF1/CK (DFF)	0.00	10.12 r

clock uncertainty	-0.30	9.82
library setup time	-0.03	9.79
data required time		9.79

data required time		9.79
data arrival time		-0.33

slack (MET)		9.46

Note that the edge at the clock pin of the flip-flop (called the **active edge**) remains unchanged. It can only be a rising or falling active edge, depending upon whether the flip-flop is rising-edge triggered or falling-edge triggered respectively.

What is *clock source latency*? This is also called *insertion delay* and is the time it takes for a clock to propagate from its source to the clock definition point of the design under analysis as depicted in Figure 8-3. This corresponds to the latency of the clock tree that is outside of the design. For example, if this design were part of a larger block, the clock source latency specifies the delay of the clock tree up to the clock pin of the design under analysis. This latency can be explicitly specified using the `set_clock_latency` command.

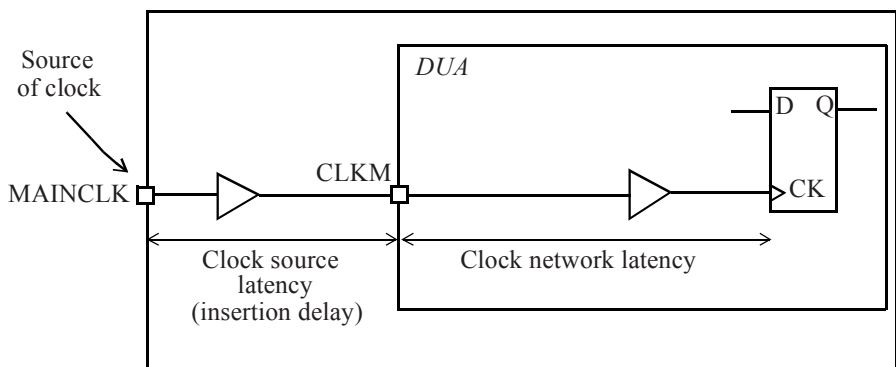


Figure 8-3 *The two types of clock latencies.*

```
set_clock_latency -source -rise 0.7 [get_clocks CLKM]
set_clock_latency -source -fall 0.65 [get_clocks CLKM]
```

In the absence of such a command, a latency of 0 is assumed. That was the assumption used in earlier path reports. Note that the source latency does not affect paths that are internal to the design and have the same launch clock and capture clock. This is because the same latency gets added to both the launch clock path and the capture clock path. However this latency does impact timing paths that go through the inputs and outputs of the design under analysis.

Without the `-source` option, the `set_clock_latency` command defines the *clock network latency* - this is the latency from the clock definition point in the DUA to the clock pin of a flip-flop. The clock network latency is used to model the delay through the clock path before the clock trees are built, that is, prior to clock tree synthesis. Once a clock tree is built and is marked as *propagated*, this clock network latency specification is ignored. The `set_clock_latency` command can be used to model the delay from the master clock to one of its generated clocks as described in Section 7.3. This command is also used to model off-chip clock latency when clock generation logic is not part of the design.

8.1.2 Input to Flip-flop Path

Here is an example path report through an input port to a flip-flop. Figure 8-4 shows the schematic related to the input path and the clock waveforms.

```
Startpoint: INA (input port clocked by VIRTUAL_CLKM)
Endpoint: UFF2 (rising edge-triggered flip-flop clocked by CLKM)
Path Group: CLKM
Path Type: max
```

Point	Incr	Path

clock VIRTUAL_CLKM (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
input external delay	2.55	2.55 f

INA (in) <-	0.00	2.55 f
UINV1/ZN (INV)	0.02	2.58 r
UAND0/Z (AN2)	0.06	2.63 r
UINV2/ZN (INV)	0.02	2.65 f
UFF2/D (DFF)	0.00	2.65 f
data arrival time		2.65
clock CLKM (rise edge)	10.00	10.00
clock source latency	0.00	10.00
CLKM (in)	0.00	10.00 r
UCKBUF0/C (CKB)	0.06	10.06 r
UCKBUF2/C (CKB)	0.07	10.12 r
UCKBUF3/C (CKB)	0.06	10.18 r
UFF2/CK (DFF)	0.00	10.18 r
clock uncertainty	-0.30	9.88
library setup time	-0.03	9.85
data required time		9.85

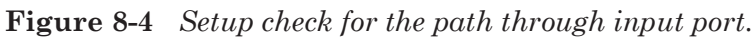
data required time		9.85
data arrival time		-2.65

slack (MET)		7.20

The first thing to notice is *input port clocked by VIRTUAL_CLKM*. As discussed in Section 7.9, this clock can be considered as an imaginary (virtual) flip-flop outside of the design that is driving the input port *INA* of the design. The clock of this virtual flip-flop is *VIRTUAL_CLKM*. In addition, the max delay from the clock pin of this virtual flip-flop to the input port *INA* is specified as 2.55ns - this appears as *input external delay* in the report. Both of these parameters are specified using the following SDC commands.

```
create_clock -name VIRTUAL_CLKM -period 10 -waveform {0 5}
set_input_delay -clock VIRTUAL_CLKM \
    -max 2.55 [get_ports INA]
```

Notice that the definition of the virtual clock *VIRTUAL_CLKM* does not have any pin from the design associated with it; this is because it is considered to be defined outside of the design (it is *virtual*). The input delay spec-



The input path starts from the port *INA*; how does one compute the delay of the first cell *UINV1* connected to port *INA*? One way to accomplish this is by specifying the driving cell of the input port *INA*. This driving cell is used to determine the drive strength and thus the slew on the port *INA*, which is then used to compute the delay of the cell *UINV1*. In the absence of any slew specification on the input port *INA*, the transition at the port is assumed to be ideal, which corresponds to a transition time of 0ns.

```
set_driving_cell -lib_cell BUFF \  
-library lib0131wc [get_ports INA]
```

Figure 8-4 also shows how the setup check is done. The time by which data must arrive at *UFF2/D* is 9.85ns. However, the data arrives at 2.65ns, thus the report shows a positive slack of 7.2ns on this path.

Input Path with Actual Clock

Input arrival times can be specified with respect to an actual clock also; these do not necessarily have to be specified with respect to a virtual clock. Examples of actual clocks are clocks on internal pins in the design, or on input ports. Figure 8-5 depicts an example where the input constraint on port *CIN* is specified relative to a clock on input port *CLKP*. This constraint is specified as:

```
set_input_delay -clock CLKP -max 4.3 [get_ports CIN]
```

Here is the input path report corresponding to this specification.

```
Startpoint: CIN (input port clocked by CLKP)  
Endpoint: UFF4 (rising edge-triggered flip-flop clocked by CLKP)  
Path Group: CLKP  
Path Type: max
```

Point	Incr	Path
-----		-----
clock CLKP (rise edge)	0.00	0.00
clock network delay (propagated)	0.00	0.00
input external delay	4.30	4.30 f
CIN (in)	0.00	4.30 f
UBUF5/Z (BUFF)	0.06	4.36 f
UXOR1/Z (XOR2)	0.10	4.46 r
UFF4/D (DFF)	0.00	4.46 r
data arrival time		4.46
clock CLKP (rise edge)	12.00	12.00
clock source latency	0.00	12.00

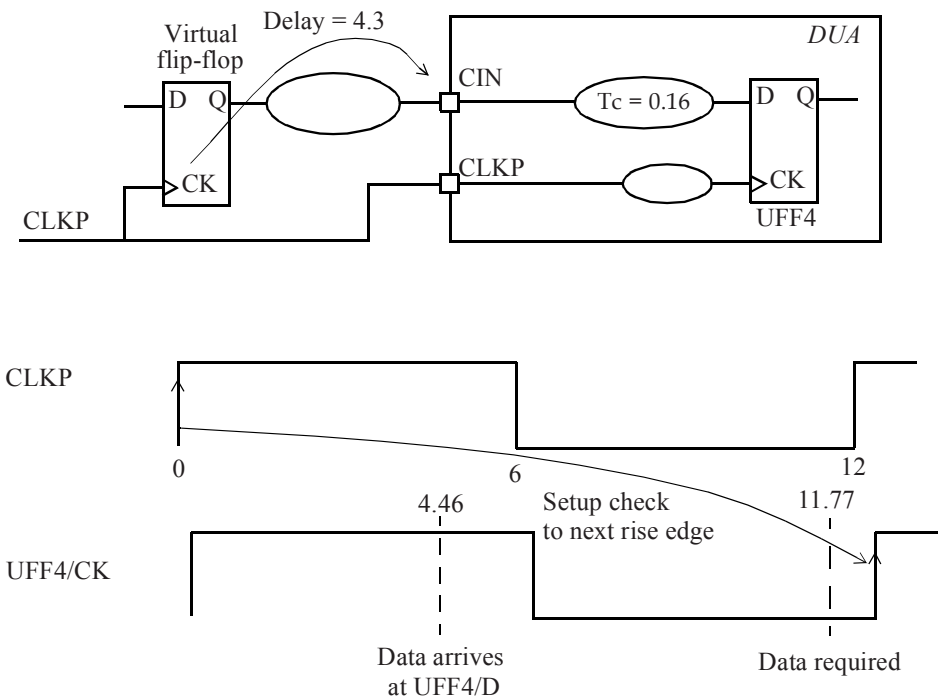


Figure 8-5 Path through input port using core clock.

CLKP (in)	0.00	12.00 r
UCKBUF4/C (CKB)	0.06	12.06 r
UCKBUF5/C (CKB)	0.06	12.12 r
UFF4/CK (DFF)	0.00	12.12 r
clock uncertainty	-0.30	11.82
library setup time	-0.05	11.77
data required time		11.77

data required time		11.77
data arrival time		-4.46

slack (MET)		7.31

Notice that the *Startpoint* specifies the reference clock for the input port to be *CLKP* as expected.

8.1.3 Flip-flop to Output Path

Similar to the input port constraint described above, an output port can be constrained either with respect to a virtual clock, or an internal clock of the design, or an input clock port, or an output clock port. Here is an example that shows the output pin *ROUT* constrained with respect to a virtual clock. The output constraint is as follows:

```
set_output_delay -clock VIRTUAL_CLKP \  
-max 5.1 [get_ports ROUT]  
set_load 0.02 [get_ports ROUT]
```

To determine the delay of the last cell connected to the output port correctly, one needs to specify the load on this port. The output load is specified above using the *set_load* command. Note that the port *ROUT* may have load contribution internal to the *DUA* and the *set_load* specification provides the additional load, which is the load contribution from outside the *DUA*. In the absence of the *set_load* specification, a value of 0 for the external load is assumed (which may not be realistic as this design would most probably be used in some other design). Figure 8-6 shows the timing path to the virtual flip-flop that has the virtual clock.

The path report through the output port is shown next.

```
Startpoint: UFF4 (rising edge-triggered flip-flop clocked by CLKP)  
Endpoint: ROUT (output port clocked by VIRTUAL_CLKP)  
Path Group: VIRTUAL_CLKP  
Path Type: max
```

Point	Incr	Path

clock CLKP (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKP (in)	0.00	0.00 r

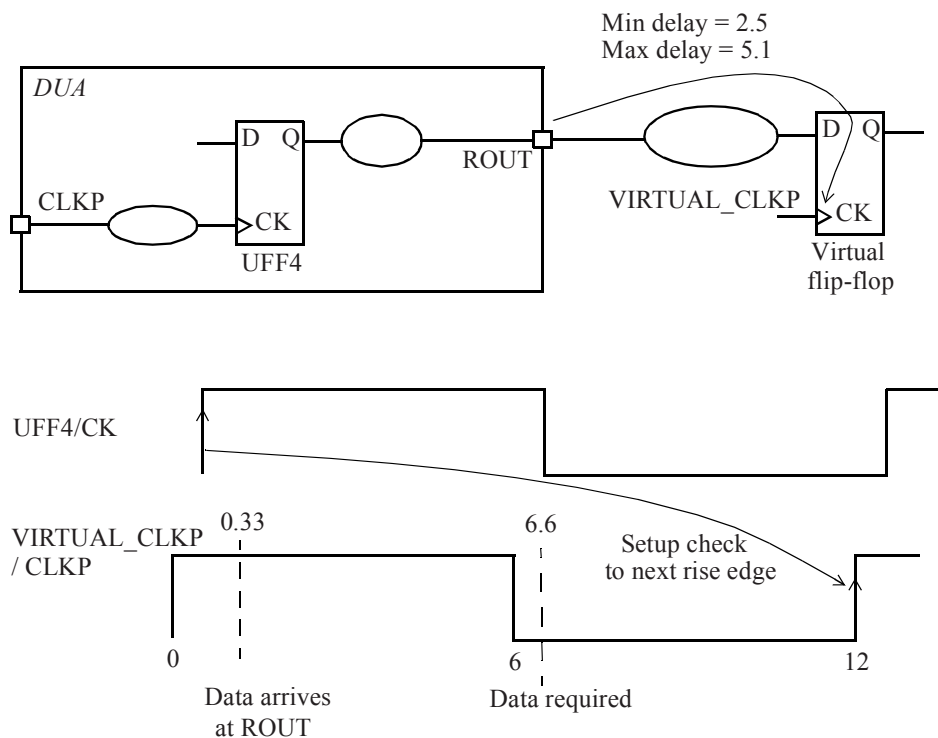


Figure 8-6 Setup check for path through output port.

UCKBUF4/C (CKB)	0.06	0.06 r
UCKBUF5/C (CKB)	0.06	0.12 r
UFF4/CK (DFF)	0.00	0.12 r
UFF4/Q (DFF)	0.13	0.25 r
UBUF3/Z (BUFF)	0.09	0.33 r
ROUT (out)	0.00	0.33 r
data arrival time		0.33
clock VIRTUAL_CLKP (rise edge)	12.00	12.00
clock network delay (ideal)	0.00	12.00
clock uncertainty	-0.30	11.70
output external delay	-5.10	6.60
data required time		6.60

data required time	6.60
data arrival time	-0.33

slack (MET)	6.27

Notice that the output delay specified appears as *output external delay* and behaves like a required setup time for the virtual flip-flop.

8.1.4 Input to Output Path

The design can have a combinational path going from an input port to an output port. This path can be constrained and timed just like the input and output paths we saw earlier. Figure 8-7 shows an example of such a path. Virtual clocks are used to specify constraints on both input and output ports.

Here are the input and output delay specifications.

```
set_input_delay -clock VIRTUAL_CLKM \  
-max 3.6 [get_ports INB]  
set_output_delay -clock VIRTUAL_CLKM \  
-max 5.8 [get_ports POUT]
```

Here is a path report that goes through the combinational logic from input *INB* to output *POUT*. Notice that any internal clock latencies, if present, have no effect on the path report.

```
Startpoint: INB (input port clocked by VIRTUAL_CLKM)  
Endpoint: POUT (output port clocked by VIRTUAL_CLKM)  
Path Group: VIRTUAL_CLKM  
Path Type: max
```

Point	Incr	Path

clock VIRTUAL_CLKM (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00

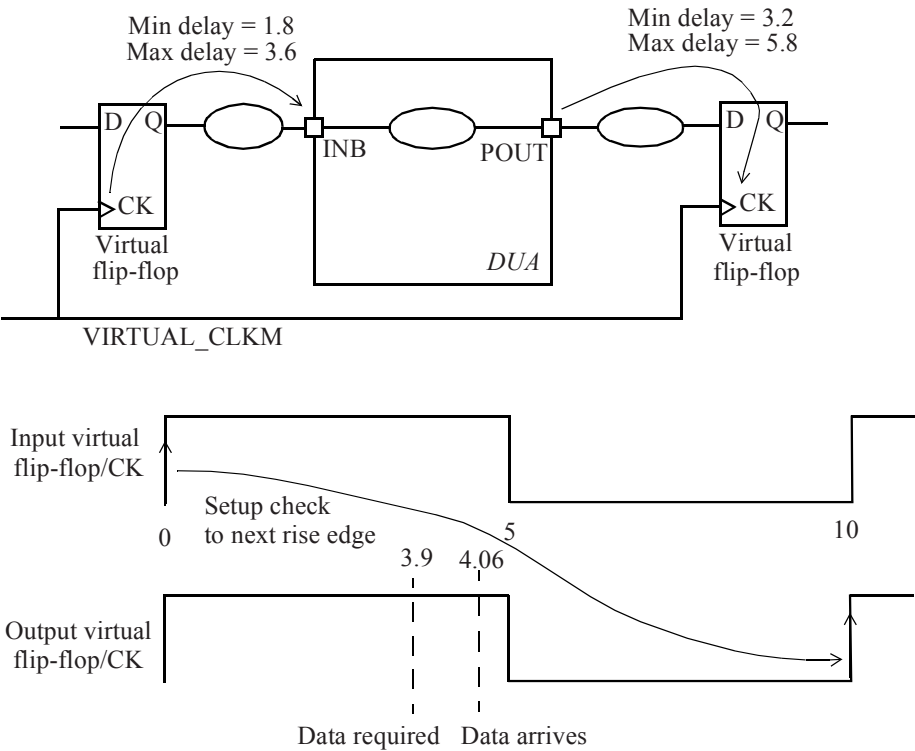


Figure 8-7 *Combinational path from input to output port.*

input external delay	3.60	3.60 f
INB (in) <-	0.00	3.60 f
UBUF0/Z (BUFF)	0.05	3.65 f
UBUF1/Z (BUFF)	0.06	3.72 f
UINV3/ZN (INV)	0.34	4.06 r
POUT (out)	0.00	4.06 r
data arrival time		4.06
clock VIRTUAL_CLKM (rise edge)	10.00	10.00
clock network delay (ideal)	0.00	10.00

clock uncertainty	-0.30	9.70
output external delay	-5.80	3.90
data required time		3.90

data required time		3.90
data arrival time		-4.06

slack (VIOLATED)		-0.16

8.1.5 Frequency Histogram

If one were to plot a frequency histogram of setup slack versus number of paths for a typical design, it would look like the one shown in Figure 8-8. Depending upon the state of the design, whether it has been optimized or not, the zero slack line would be more towards the right for an unoptimized design and more towards the left for an optimized design. For a design that has zero violations, that is no paths with negative slack, the entire curve would be to the right of the zero slack line.

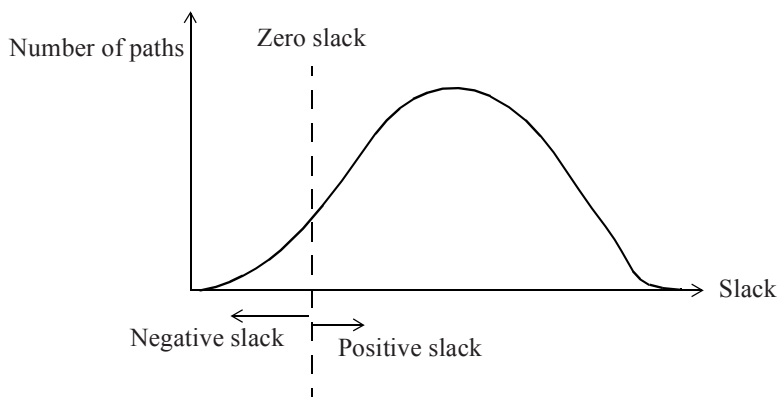


Figure 8-8 *Frequency histogram of timing slack in paths.*

Here is a histogram shown in a textual form that can often be produced by a static timing analysis tool.

{ -INF	375	0}
{ 375	380	237}
{ 380	385	425}
{ 385	390	1557}
{ 390	395	1668}
{ 395	400	1559}
{ 400	405	1244}
{ 405	410	1079}
{ 410	415	941}
{ 415	420	431}
{ 420	425	404}
{ 425	430	1}
{ 430	+INF	0}

The first two indices denote the slack range and the third index is the number of paths within that slack range, for example, there are 941 paths with slack in the range of 410ps to 415ps. The histogram indicates that this design has no failing paths, that is all paths have positive slack, and that the most critical path has a positive slack between 375ps and 380ps.

Designs that are tough to meet timing would have their hump of the histogram more towards the left, that is, have many paths with slack closer to zero. One other observation that can be made by looking at a frequency histogram is on the ability to further optimize the design to achieve zero slack, that is, how difficult it is to close timing. If the number of failing paths is small and the negative slack is also small, the design is relatively close to meeting the required timing. However, if the number of failing paths is large and the negative slack magnitude is also large, this implies that the design would require a lot of effort to meet the required timing.

8.2 Hold Timing Check

A **hold timing check** ensures that a flip-flop output value that is changing does not pass through to a capture flip-flop and overwrite its output before the flip-flop has had a chance to capture its original value. This check is based on the hold requirement of a flip-flop. The hold specification of a flip-flop requires that the data being latched should be held stable for a specified amount of time after the active edge of the clock. Figure 8-9 shows the hold requirement of a typical flip-flop.

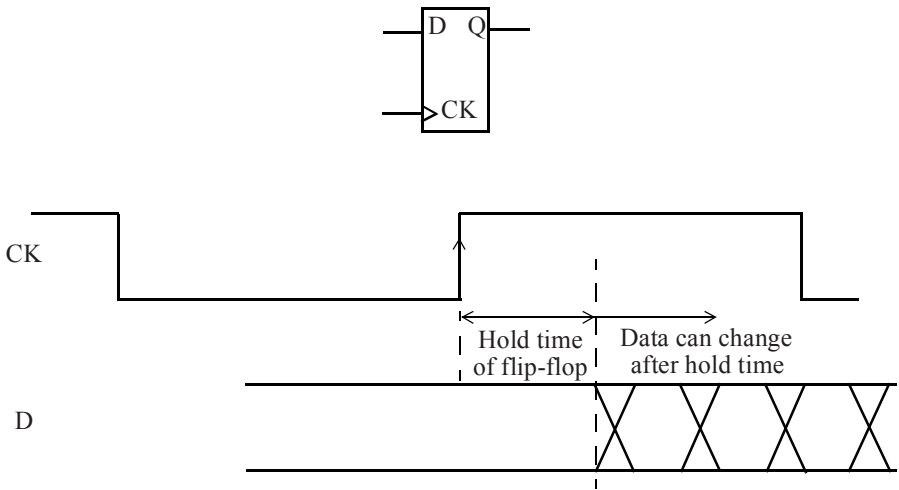


Figure 8-9 *Hold requirement of a flip-flop.*

Just like the setup check, a hold timing check is between the launch flip-flop - the flip-flop that launches the data, and the capture flip-flop - the flip-flop that captures the data and whose hold time must be satisfied. The clocks to these two flip-flops can be the same or can be different. The hold check is from one active edge of the clock in the launch flip-flop to the same clock edge at the capture flip-flop. Thus, a hold check is independent of the clock period. The hold check is carried out on each active edge of the clock of the capture flip-flop.

We now examine a simple example, shown in Figure 8-10, where both the launch and the capture flip-flops have the same clock.

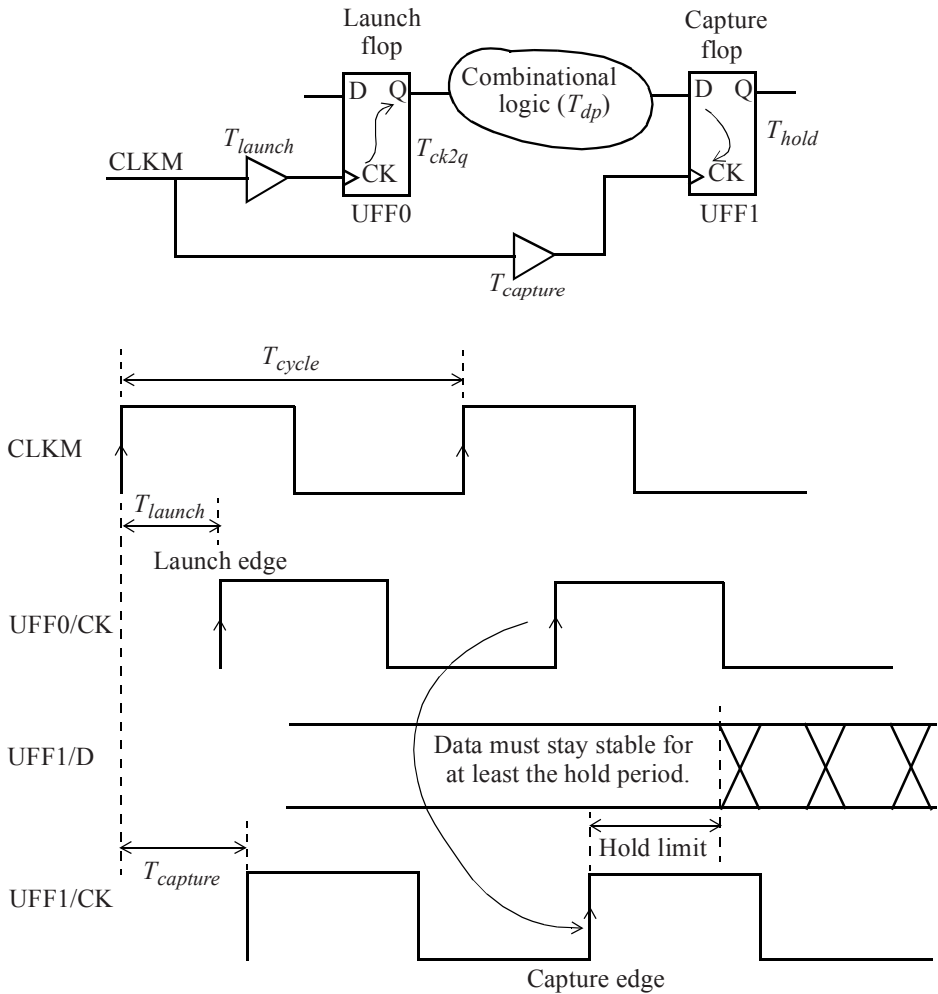


Figure 8-10 Data and clock signals for hold timing check.

Consider the second rising edge of clock $CLKM$. The data launched by the rising edge of the clock takes $T_{launch} + T_{ck2q} + T_{dp}$ time to get to the D pin of the capture flip-flop $UFF1$. The same edge of the clock takes $T_{capture}$ time to get to the clock pin of the capture flip-flop. The intention is for the data from the launch flip-flop to be captured by the capture flip-flop in the next clock cycle. If the data is captured in the same clock cycle, the intended data in the capture flip-flop (from the previous clock cycle) is overwritten. The hold time check is to ensure that the intended data in the capture flip-flop is not overwritten. The hold time check verifies that the difference between these two times (data arrival time and clock arrival time at capture flip-flop) must be larger than the hold time of the capture flip-flop, so that the previous data on the flip-flop is not overwritten and the data is reliably captured in the flip-flop.

The hold check can be mathematically expressed as:

$$T_{launch} + T_{ck2q} + T_{dp} > T_{capture} + T_{hold}$$

where T_{launch} is the delay of the clock tree of the launch flip-flop, T_{dp} is the delay in the combinational logic data path and $T_{capture}$ is the delay on the clock tree for the capture flip-flop. In other words, the total time required for data launched by a clock edge to arrive at the D pin of the capture flip-flop must be larger than the time required for the same edge of the clock to travel to the capture flip-flop plus the hold time. This ensures that $UFF1/D$ remains stable until the hold time of the flip-flop after the rising edge of the clock on its clock pin $UFF1/CK$.

The hold checks impose a lower bound or min constraint for paths to the data pin on the capture flip-flop; the fastest path to the D pin of the capture flip-flop needs to be determined. This implies that the hold checks are *always* verified using the shortest paths. Thus, the hold checks are typically performed at the fast timing corner.

Even when there is only one clock in the design, the clock tree can result in the arrival times of the clocks at the launch and capture flip-flops to be substantially different. To ensure reliable data capture, the clock edge at the

capture flip-flop must arrive before the data can change. A hold timing check ensures that (see Figure 8-11):

- Data from the subsequent launch edge must not be captured by the setup receiving edge.
- Data from the setup launch edge must not be captured by the preceding receiving edge.

These two hold checks are essentially the same if both the launch and capture clock belong to the same clock domain. However, when the launch and capture clocks are at different frequencies or in different clock domains, the above two conditions may map into different constraints. In such cases, the worst hold check is the one that is reported. Figure 8-11 shows these two checks pictorially.

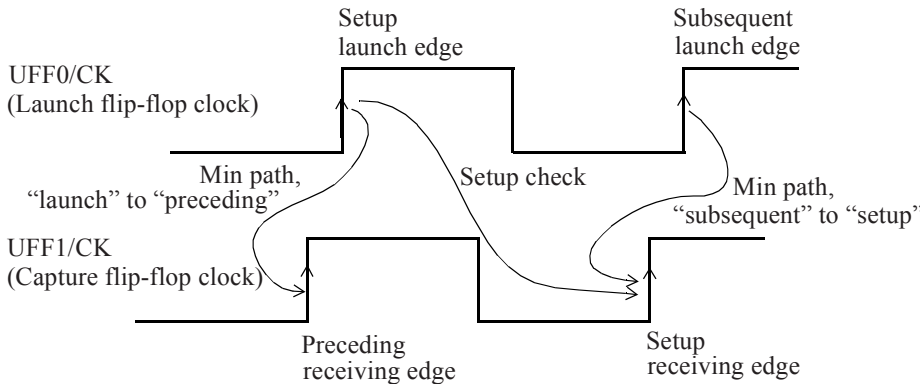


Figure 8-11 *Two hold checks for one setup check.*

UFF0 is the launch flip-flop and UFF1 is the capture flip-flop. The setup check is between the *setup launch edge* and the *setup receiving edge*. The *subsequent launch edge* must not propagate data so fast that the *setup receiving edge* does not have time to capture its data reliably. In addition, the *setup launch edge* must not propagate data so fast that the *preceding receiving edge* does not get a chance to capture its data. The worst hold check corresponds

to the most restrictive hold check amongst various scenarios described above.

More general clocking such as for multicycle paths and multi-frequency paths are discussed later in Section 8.3 and Section 8.8 respectively. The discussion covers the relationships between setup checks and hold checks, especially, how hold checks are derived from the setup check relationships. While setup violations can cause the operating frequency of the design to be lowered, the hold violations can kill a design, that is, make the design inoperable at any frequency. Thus it is very important to understand the hold timing checks and resolve any violations.

8.2.1 Flip-flop to Flip-flop Path

This section illustrates the flip-flop to flip-flop hold path based upon the example depicted in Figure 8-2. Here is the path report of a hold timing check for the example from Section 8.1 for the setup check path.

Startpoint: UFF0 (rising edge-triggered flip-flop clocked by CLKM)
Endpoint: UFF1 (rising edge-triggered flip-flop clocked by CLKM)
Path Group: CLKM
Path Type: min

Point	Incr	Path
-----		-----
clock CLKM (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKM (in)	0.00	0.00 r
UCKBUF0/C (CKB)	0.06	0.06 r
UCKBUF1/C (CKB)	0.06	0.11 r
UFF0/CK (DFF)	0.00	0.11 r
UFF0/Q (DFF) <-	0.14	0.26 r
UNOR0/ZN (NR2)	0.02	0.28 f
UBUF4/Z (BUFF)	0.06	0.33 f
UFF1/D (DFF)	0.00	0.33 f
data arrival time		0.33
 clock CLKM (rise edge)	 0.00	 0.00
clock source latency	0.00	0.00

CLKM (in)	0.00	0.00 r
UCKBUF0/C (CKB)	0.06	0.06 r
UCKBUF2/C (CKB)	0.07	0.12 r
UFF1/CK (DFF)	0.00	0.12 r
clock uncertainty	0.05	0.17
library hold time	0.01	0.19
data required time		0.19

data required time		0.19
data arrival time		-0.33

slack (MET)		0.14

Notice that the *Path Type* is described as *min* indicating that the cell delay values along the shortest path are used which corresponds to hold timing checks. The *library hold time* specifies the hold time of flip-flop *UFF1*. (The hold time for flip-flops can also be negative as explained earlier in Section 3.4.) Notice that both the capture and receive timing is computed at the rising edge (active edge of flip-flop) of clock *CLKM*. The timing report shows that the earliest time the new data can arrive at *UFF1* while enabling the previous data to be safely captured is 0.19ns. Since the new data arrives at 0.33ns, the report shows a positive hold slack of 0.14ns.

Figure 8-12 shows the times of the clock signals at the launch and capture flip-flops along with the earliest allowed and actual arrival time for the data at the capture flip-flop. Since the data arrives later than the data required time (which for hold is the earliest allowed), the hold condition is met.

Hold Slack Calculation

An interesting point to note is the difference in the way the slack is computed for setup and hold timing reports. In the setup timing reports, the arrival time and the required time are computed and the slack is computed to be the required time minus arrival time. However in the hold timing reports, when we compute required time minus arrival time, a negative re-

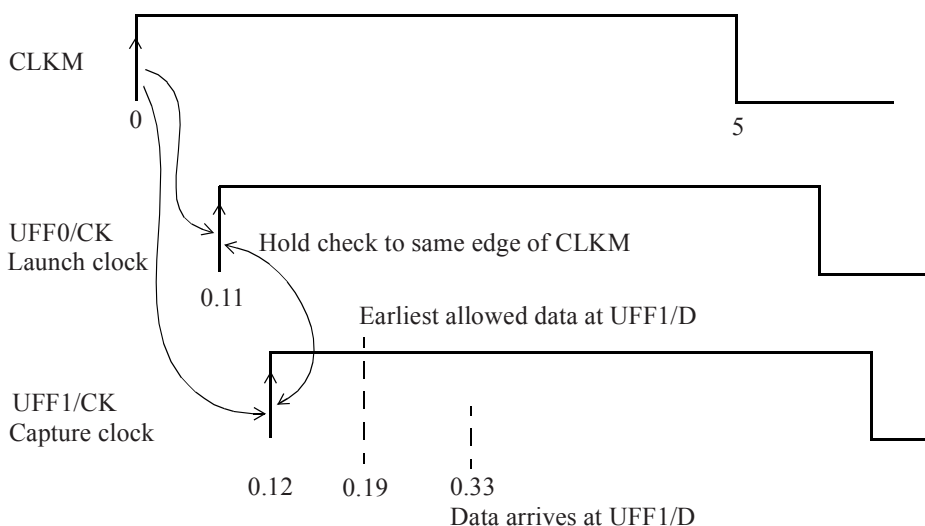


Figure 8-12 Clock waveforms for hold timing check on internal path.

sult translates into a positive slack (means hold constraint is satisfied), while a positive result translates into a negative slack (means hold constraint is not satisfied).

8.2.2 Input to Flip-flop Path

The hold timing check from an input port is described next. See Figure 8-4 for an example. The min delay on the input port is specified using a virtual clock as:

```
set_input_delay -clock VIRTUAL_CLKM \
-min 1.1 [get_ports INA]
```

Here is the hold timing report.

```
Startpoint: INA (input port clocked by VIRTUAL_CLKM)
Endpoint: UFF2 (rising edge-triggered flip-flop clocked by CLKM)
```

Path Group: CLKM

Path Type: min

Point	Incr	Path

clock VIRTUAL_CLKM (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
input external delay	1.10	1.10 f
INA (in) <-	0.00	1.10 f
UINV1/ZN (INV)	0.02	1.13 r
UAND0/Z (AN2)	0.06	1.18 r
UINV2/ZN (INV)	0.02	1.20 f
UFF2/D (DFF)	0.00	1.20 f
data arrival time		1.20
clock CLKM (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKM (in)	0.00	0.00 r
UCKBUF0/C (CKB)	0.06	0.06 r
UCKBUF2/C (CKB)	0.07	0.12 r
UCKBUF3/C (CKB)	0.06	0.18 r
UFF2/CK (DFF)	0.00	0.18 r
clock uncertainty	0.05	0.23
library hold time	0.01	0.25
data required time		0.25

data required time		0.25
data arrival time		-1.20

slack (MET)		0.95

The *set_input_delay* appears as *input external delay*. The hold check is done at time 0 between rising edge of *VIRTUAL_CLKM* and rising edge of *CLKM*. The required arrival time for data to be captured by *UFF2* without violating its hold time is 0.25ns - this indicates that the data should arrive after 0.25ns. Since the data can arrive only at 1.2ns, this shows a positive slack of 0.95ns.

8.2.3 Flip-flop to Output Path

Here is a hold timing check at an output port. See Figure 8-6 for the example. The output port specification appears as:

```
set_output_delay -clock VIRTUAL_CLKP \  
-min 2.5 [get_ports ROUT]
```

The output delay is specified with respect to a virtual clock. Here is the hold report.

Startpoint: UFF4 (**rising edge-triggered flip-flop** clocked by CLKP)
Endpoint: ROUT (**output port** clocked by VIRTUAL_CLKP)
Path Group: VIRTUAL_CLKP
Path Type: **min**

Point	Incr	Path

clock CLKP (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKP (in)	0.00	0.00 r
UCKBUF4/C (CKB)	0.06	0.06 r
UCKBUF5/C (CKB)	0.06	0.12 r
UFF4/CK (DFF)	0.00	0.12 r
UFF4/Q (DFF)	0.13	0.25 f
UBUF3/Z (BUFF)	0.08	0.33 f
ROUT (out)	0.00	0.33 f
data arrival time		0.33
clock VIRTUAL_CLKP (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
clock uncertainty	0.05	0.05
output external delay	-2.50	-2.45
data required time		-2.45

data required time		-2.45
data arrival time		-0.33

slack (MET)		2.78

Notice that the *set_output_delay* appears as *output external delay*.

Flip-flop to Output Path with Actual Clock

Here is a path report of a hold timing check to an output port. See Figure 8-13. The output min delay is specified with respect to a real clock.

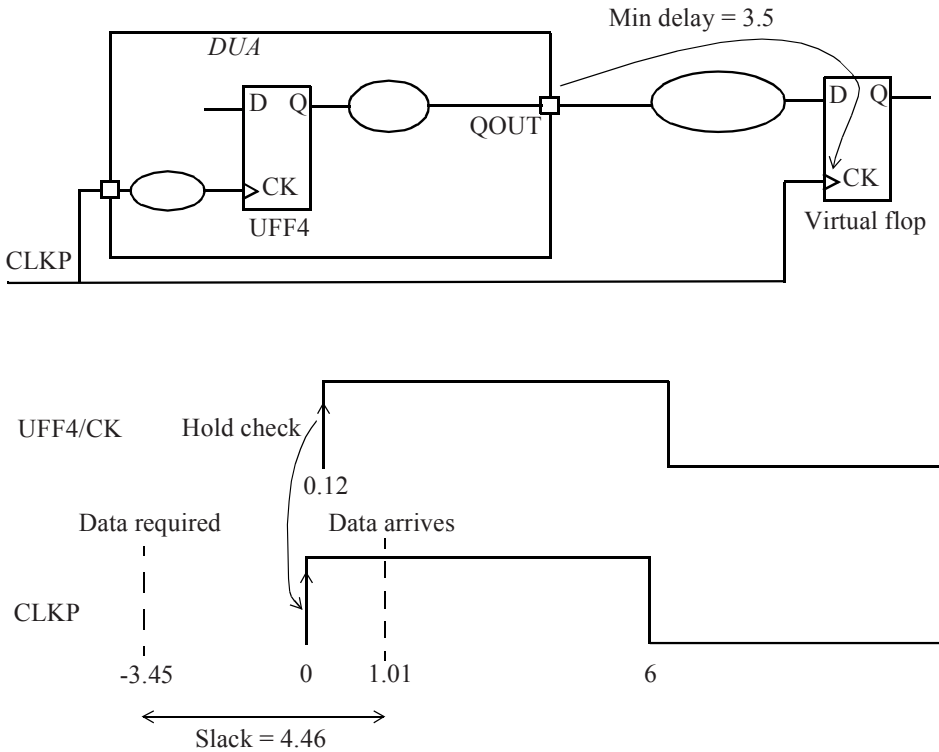


Figure 8-13 Path through output port.

```
set_output_delay -clock CLKP -min 3.5 [get_ports QOUT]
set_load 0.55 [get_ports QOUT]
```

Here is the hold timing report.

Startpoint: UFF4 (**rising edge-triggered flip-flop** clocked by CLKP)
Endpoint: QOUT (**output port** clocked by CLKP)
Path Group: CLKP
Path Type: **min**

Point	Incr	Path

clock CLKP (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKP (in)	0.00	0.00 r
UCKBUF4/C (CKB)	0.06	0.06 r
UCKBUF5/C (CKB)	0.06	0.12 r
UFF4/CK (DFF)	0.00	0.12 r
UFF4/Q (DFF)	0.14	0.26 r
UINV4/ZN (INV)	0.75	1.01 f
QOUT (out)	0.00	1.01 f
data arrival time		1.01
clock CLKP (rise edge)	0.00	0.00
clock network delay (propagated)	0.00	0.00
clock uncertainty	0.05	0.05
output external delay	-3.50	-3.45
data required time		-3.45

data required time		-3.45
data arrival time		-1.01

slack (MET)		4.46

The hold timing check is performed on the rising edge (active edge of flip-flop) of clock *CLKP*. The above report indicates that the flip-flop to output has a positive slack of 4.46ns for the hold time.

8.2.4 Input to Output Path

Here is a hold timing check on an input to output path shown in Figure 8-7. The specifications on the ports are:

```
set_load -pin_load 0.15 [get_ports POUT]
set_output_delay -clock VIRTUAL_CLKM \
  -min 3.2 [get_ports POUT]
set_input_delay -clock VIRTUAL_CLKM \
  -min 1.8 [get_ports INB]
set_input_transition 0.8 [get_ports INB]
```

Startpoint: INB (**input port** clocked by VIRTUAL_CLKM)
 Endpoint: POUT (**output port** clocked by VIRTUAL_CLKM)
 Path Group: VIRTUAL_CLKM
 Path Type: **min**

Point	Incr	Path

clock VIRTUAL_CLKM (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
input external delay	1.80	1.80 r
INB (in) <-	0.00	1.80 r
UBUF0/Z (BUFF)	0.04	1.84 r
UBUF1/Z (BUFF)	0.06	1.90 r
UINV3/ZN (INV)	0.22	2.12 f
POUT (out)	0.00	2.12 f
data arrival time		2.12
clock VIRTUAL_CLKM (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
clock uncertainty	0.05	0.05
output external delay	-3.20	-3.15
data required time		-3.15

data required time		-3.15
data arrival time		-2.12

slack (MET)		5.27

The specification on the output port is specified with respect to a virtual clock and hence the hold check is performed on the rising (default active) edge of that virtual clock.

8.3 Multicycle Paths

In some cases, the combinational data path between two flip-flops can take more than one clock cycle to propagate through the logic. In such cases, the combinational path is declared as a **multicycle path**. Even though the data is being captured by the capture flip-flop on every clock edge, we direct STA that the relevant capture edge occurs after the specified number of clock cycles.

Figure 8-14 shows an example. Since the data path can take up to three clock cycles, a setup multicycle check of three cycles should be specified. The multicycle setup constraints specified to achieve this are given below.

```
create_clock -name CLKM -period 10 [get_ports CLKM]
set_multicycle_path 3 -setup \
  -from [get_pins UFF0/Q] \
  -to [get_pins UFF1/D]
```

The setup multicycle constraint specifies that the path from *UFF0/CK* to *UFF1/D* can take up to three clock cycles to complete for a setup check. This implies that the design utilizes the required data from *UFF1/Q* only every third cycle instead of every cycle.

Here is a setup path report that has the multicycle constraint specified.

```
Startpoint: UFF0 (rising edge-triggered flip-flop clocked by CLKM)
Endpoint: UFF1 (rising edge-triggered flip-flop clocked by CLKM)
Path Group: CLKM
Path Type: max
```

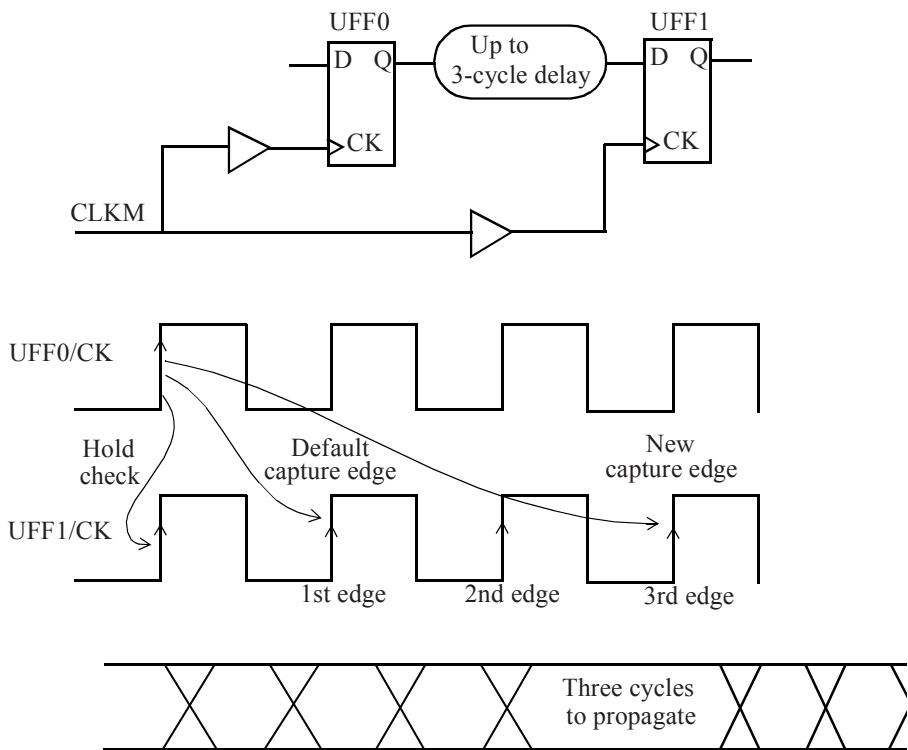


Figure 8-14 A three-cycle multicycle path.

Point	Incr	Path
clock CLKM (rise edge)	0.00	0.00
clock network delay (propagated)	0.11	0.11
UFF0/CK (DFF)	0.00	0.11 r
UFF0/Q (DFF) <=	0.14	0.26 f
UNOR0/ZN (NR2)	0.04	0.30 r
UBUF4/Z (BUFF)	0.05	0.35 r
UFF1/D (DFF)	0.00	0.35 r
data arrival time		0.35

clock CLKM (rise edge)	30.00	30.00
clock network delay (propagated)	0.12	30.12
clock uncertainty	-0.30	29.82
UFF1/CK (DFF)		29.82 r
library setup time	-0.04	29.78
data required time		29.78

data required time		29.78
data arrival time		-0.35

slack (MET)		29.43

Notice that the clock edge for the capture flip-flop is now three clock cycles away, at 30ns.

We now examine the hold timing check on the example multicycle path. In most common scenarios, we would want the hold check to stay as it was in a single cycle setup case, which is the one shown in the Figure 8-14. This ensures that the data is free to change anywhere in between the three clock cycles. A hold multicycle of two is specified to get the same behavior of a hold check as in a single cycle setup case. This is because in the absence of such a hold multicycle specification, the default hold check is done on the active edge prior to the setup capture edge which is not the intent. We need to move the hold check two cycles prior to the default hold check edge and hence a hold multicycle of two is specified. The intended behavior is shown in Figure 8-15. With the multicycle hold, the min delay for the data path can be less than one clock cycle.

```
set_multicycle_path 2 -hold -from [get_pins UFF0/Q] \  
-to [get_pins UFF1/D]
```

The number of cycles denoted on a multicycle hold specifies how many clock cycles to move back from its default hold check edge (which is one active edge prior to the setup capture edge). Here is the path report.

```
Startpoint: UFF0 (rising edge-triggered flip-flop clocked by CLKP)  
Endpoint: UFF1 (rising edge-triggered flip-flop clocked by CLKP)
```

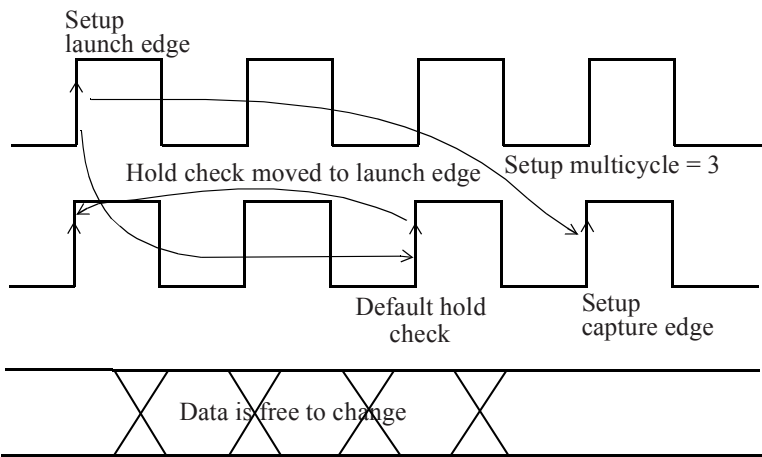


Figure 8-15 *Hold check moved back to launch edge.*

Path Group: CLKP
Path Type: min

Point	Incr	Path
clock CLKP (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKP (in)	0.00	0.00 r
UCKBUF4/C (CKB)	0.07	0.07 r
UFF0/CK (DFF)	0.00	0.07 r
UFF0/Q (DFF) <=	0.15	0.22 f
UXOR1/Z (XOR2)	0.07	0.29 f
UFF1/D (DFF)	0.00	0.29 f
data arrival time		0.29
 clock CLKP (rise edge)	 0.00	 0.00
clock source latency	0.00	0.00
CLKP (in)	0.00	0.00 r
UCKBUF4/C (CKB)	0.07	0.07 r
UCKBUF5/C (CKB)	0.06	0.13 r
UFF1/CK (DFF)	0.00	0.13 r
clock uncertainty	0.05	0.18
library hold time	0.01	0.19

data required time	0.19

data required time	0.19
data arrival time	-0.29

slack (MET)	0.11

Since this path has a setup multicycle of three, its default hold check is on the active edge prior to the capture edge. In most designs, if the max path (or setup) requires N clock cycles, it is not feasible to achieve the min path constraint to be greater than $(N-1)$ clock cycles. By specifying a multicycle hold of two cycles, the hold check edge is moved back to the launch edge (at 0ns) as shown in the above path report.

Thus, in most designs, a multicycle setup specified as N (cycles) should be accompanied by a multicycle hold constraint specified as $N-1$ (cycles).

What happens when a multicycle setup of N is specified but the corresponding $N-1$ multicycle hold is missing? In such a case, the hold check is performed on the edge one cycle prior to the setup capture edge. The case of such a hold check on a multicycle setup of 3 is shown in Figure 8-16.

This imposes a restriction that data can only change in the one cycle before the setup capture edge as the figure shows. Thus the data path must have a min delay of at least two clock cycles to meet this requirement. Here is such a path report.

Startpoint: UFF0 (rising edge-triggered flip-flop clocked by CLKM)
Endpoint: UFF1 (rising edge-triggered flip-flop clocked by CLKM)
Path Group: CLKM
Path Type: **min**

Point	Incr	Path

clock CLKM (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKM (in)	0.00	0.00 r
UCKBUF0/C (CKB)	0.06	0.06 r

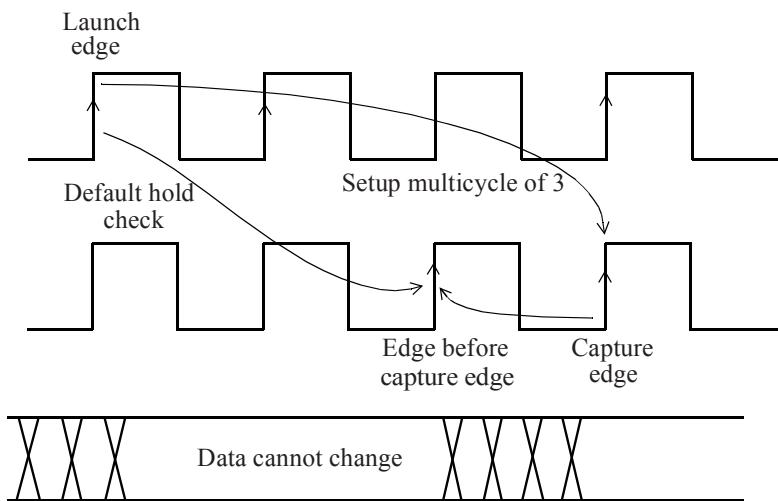


Figure 8-16 *Default hold check in a multicycle path.*

UCKBUF1/C (CKB)	0.06	0.11 r
UFF0/CK (DFF)	0.00	0.11 r
UFF0/Q (DFF) <-	0.14	0.26 r
UNOR0/ZN (NR2)	0.02	0.28 f
UBUF4/Z (BUFF)	0.06	0.33 f
UFF1/D (DFF)	0.00	0.33 f
data arrival time		0.33
clock CLKM (rise edge)	20.00	20.00
clock source latency	0.00	20.00
CLKM (in)	0.00	20.00 r
UCKBUF0/C (CKB)	0.06	20.06 r
UCKBUF2/C (CKB)	0.07	20.12 r
UFF1/CK (DFF)	0.00	20.12 r
clock uncertainty	0.05	20.17
library hold time	0.01	20.19
data required time		20.19

data required time		20.19
data arrival time		-0.33

```
slack (VIOLATED)
```

```
-19.85
```

Notice from the path report that the hold is being checked one clock edge prior to the capture edge, leading to a large hold violation. In effect, the hold check is requiring a minimum delay in the combinational logic of at least two clock cycles.

Crossing Clock Domains

Let us consider the case when there is a multicycle between two different clocks of the same period. (The case where the clock periods are different is described later in this chapter.)

Example 1:

```
create_clock -name CLKM \  
-period 10 -waveform {0 5} [get_ports CLKM]  
create_clock -name CLKP \  
-period 10 -waveform {0 5} [get_ports CLKP]
```

The setup multicycle multiplier represents the number of clock cycles for a given path. This is illustrated in Figure 8-17. The default setup capture edge is always one cycle away. A setup multicycle of 2 puts the capture edge two clock cycles away from the launch edge.

The hold multicycle multiplier represents the number of clock cycles before the setup capture edge that the hold check will occur, regardless of the launch edge. This is illustrated in Figure 8-18. The default hold check is one cycle before the setup capture edge. A hold multicycle specification of 1 puts the hold check one cycle prior to the default hold check, and thus becomes two cycles prior to the capture edge.

```
set_multicycle_path 2 \  
-from [get_pins UFF0/CK] -to [get_pins UFF3/D]  
# Since no -hold option is specified, the default option
```

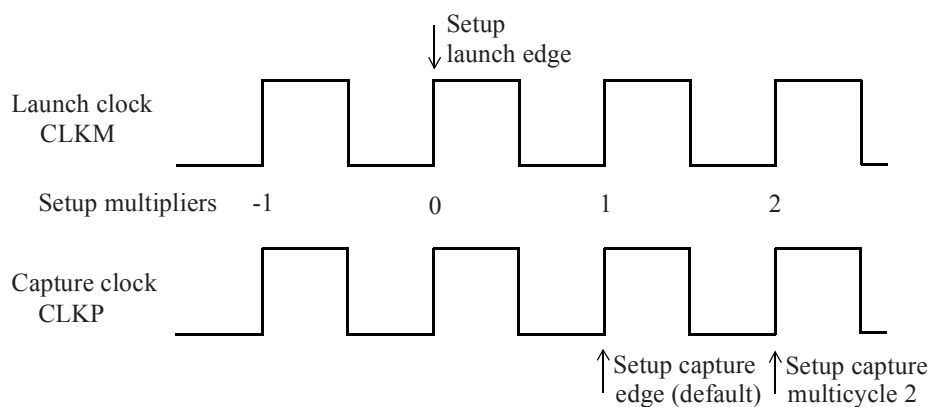



Figure 8-17 *Capture clock edges for various setup multicycle multiplier settings.*

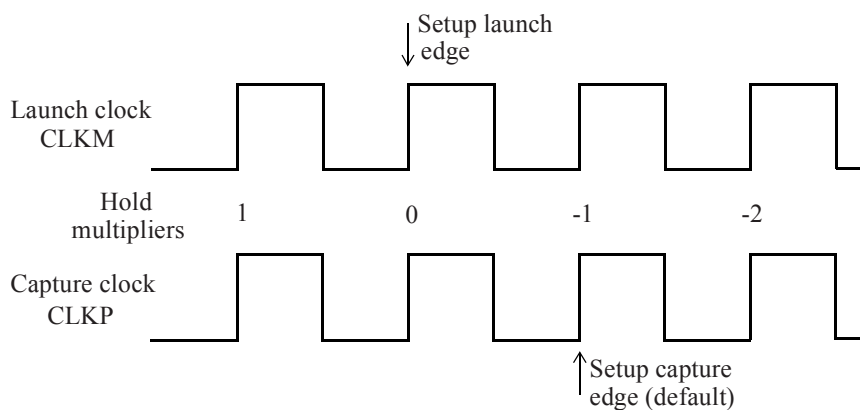


Figure 8-18 *Capture clock edges for various hold multicycle multiplier settings.*

```
# -setup is assumed. This implies that the setup
# multiplier is 2 and the hold multiplier is 0.
```

Here is the setup path report corresponding to the multicycle specification.

Startpoint: UFF0 (rising edge-triggered flip-flop clocked by **CLKM**)
Endpoint: UFF3 (rising edge-triggered flip-flop clocked by **CLKP**)
Path Group: **CLKP**
Path Type: **max**

Point	Incr	Path

clock CLKM (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKM (in)	0.00	0.00 r
UCKBUF0/C (CKB)	0.06	0.06 r
UCKBUF1/C (CKB)	0.06	0.11 r
UFF0/CK (DFF)	0.00	0.11 r
UFF0/Q (DFF) <-	0.14	0.26 f
UINV0/ZN (INV)	0.03	0.28 r
UFF3/D (DFF)	0.00	0.28 r
data arrival time		0.28
clock CLKP (rise edge)	20.00	20.00
clock source latency	0.00	20.00
CLKP (in)	0.00	20.00 r
UCKBUF4/C (CKB)	0.06	20.06 r
UFF3/CK (DFF)	0.00	20.06 r
clock uncertainty	-0.30	19.76
library setup time	-0.04	19.71
data required time		19.71

data required time		19.71
data arrival time		-0.28

slack (MET)		19.43

Note that the path group specified in a path report is always that of the capture flip-flop, in this case, *CLKP*.

Next is the hold timing check path report. The hold multiplier defaults to 0 and thus the hold check is carried out at 10ns, one clock cycle prior to the capture edge.

Startpoint: UFF0 (rising edge-triggered flip-flop clocked by **CLKM**)
 Endpoint: UFF3 (rising edge-triggered flip-flop clocked by **CLKP**)
 Path Group: CLKP
 Path Type: **min**

Point	Incr	Path

clock CLKM (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKM (in)	0.00	0.00 r
UCKBUF0/C (CKB)	0.06	0.06 r
UCKBUF1/C (CKB)	0.06	0.11 r
UFF0/CK (DFF)	0.00	0.11 r
UFF0/Q (DFF) <-	0.14	0.26 r
UINV0/ZN (INV)	0.02	0.28 f
UFF3/D (DFF)	0.00	0.28 f
data arrival time		0.28
clock CLKP (rise edge)	10.00	10.00
clock source latency	0.00	10.00
CLKP (in)	0.00	10.00 r
UCKBUF4/C (CKB)	0.06	10.06 r
UFF3/CK (DFF)	0.00	10.06 r
clock uncertainty	0.05	10.11
library hold time	0.02	10.12
data required time		10.12

data required time		10.12
data arrival time		-0.28

slack (VIOLATED)		-9.85

The above report shows the hold violation which can be removed by setting a multicycle hold of 1. This is illustrated in a separate example next.

Example II:

Another example of a multicycle specified across two different clock domains is given below.

```
set_multicycle_path 2 \  
-from [get_pins UFF0/CK] -to [get_pins UFF3/D] -setup  
set_multicycle_path 1 \  
-from [get_pins UFF0/CK] -to [get_pins UFF3/D] -hold  
# The -setup and -hold options are explicitly specified.
```

Here is the setup path timing report for the multicycle setup of 2.

```
Startpoint: UFF0 (rising edge-triggered flip-flop clocked by CLKM)  
Endpoint: UFF3 (rising edge-triggered flip-flop clocked by CLKP)  
Path Group: CLKP  
Path Type: max
```

Point	Incr	Path

clock CLKM (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKM (in)	0.00	0.00 r
UCKBUF0/C (CKB)	0.06	0.06 r
UCKBUF1/C (CKB)	0.06	0.11 r
UFF0/CK (DFF)	0.00	0.11 r
UFF0/Q (DFF) <-	0.14	0.26 f
UNAND0/ZN (ND2)	0.03	0.29 r
UFF3/D (DFF)	0.00	0.29 r
data arrival time		0.29
clock CLKP (rise edge)	20.00	20.00
clock source latency	0.00	20.00
CLKP (in)	0.00	20.00 r
UCKBUF4/C (CKB)	0.07	20.07 r
UFF3/CK (DFF)	0.00	20.07 r
clock uncertainty	-0.30	19.77
library setup time	-0.04	19.72
data required time		19.72

data required time	19.72
data arrival time	-0.29

slack (MET)	19.44

Here is the hold check timing path report for the multicycle hold of 1.

Startpoint: UFF0 (rising edge-triggered flip-flop clocked by **CLKM**)
 Endpoint: UFF3 (rising edge-triggered flip-flop clocked by **CLKP**)
 Path Group: **CLKP**
 Path Type: **min**

Point	Incr	Path

clock CLKM (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKM (in)	0.00	0.00 r
UCKBUF0/C (CKB)	0.06	0.06 r
UCKBUF1/C (CKB)	0.06	0.11 r
UFF0/CK (DFF)	0.00	0.11 r
UFF0/Q (DFF) <-	0.14	0.26 r
UNAND0/ZN (ND2)	0.03	0.29 f
UFF3/D (DFF)	0.00	0.29 f
data arrival time		0.29
clock CLKP (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKP (in)	0.00	0.00 r
UCKBUF4/C (CKB)	0.07	0.07 r
UFF3/CK (DFF)	0.00	0.07 r
clock uncertainty	0.05	0.12
library hold time	0.02	0.13
data required time		0.13

data required time		0.13
data arrival time		-0.29

slack (MET)		0.16

Note that the example reports for the setup and hold checks in this section are for the same timing corner. However, the setup checks are generally hardest to meet (have the lowest slack) at the worst-case slow corner whereas the hold checks are generally hardest to meet (have the lowest slack) at the best-case fast corner.

8.4 False Paths

It is possible that certain timing paths are not real (or not possible) in the actual functional operation of the design. Such paths can be turned off during STA by setting these as false paths. A false path is ignored by the STA for analysis.

Examples of false paths could be from one clock domain to another clock domain, from a clock pin of a flip-flop to the input of another flip-flop, through a pin of a cell, through pins of multiple cells, or a combination of these. When a false path is specified through a pin of a cell, all paths that go through that pin are ignored for timing analysis. The advantage of identifying the false paths is that the analysis space is reduced, thereby allowing the analysis to focus only on the real paths. This helps cut down the analysis time as well. However, too many false paths which are wildcarded using the *through* specification can slow down the analysis.

A false path is set using the **set_false_path** specification. Here are some examples.

```
set_false_path -from [get_clocks SCAN_CLK] \  
-to [get_clocks CORE_CLK]  
# Any path starting from the SCAN_CLK domain to the  
# CORE_CLK domain is a false path.  
  
set_false_path -through [get_pins UMUX0/S]  
# Any path going through this pin is false.
```

```
set_false_path \  
  -through [get_pins SAD_CORE/RSTN]  
# The false path specifications can also be specified to,  
# through, or from a module pin instance.  
  
set_false_path -to [get_ports TEST_REG*]  
# All paths that end in port named TEST_REG* are false paths.  
  
set_false_path -through UINV/Z -through UAND0/Z  
# Any path that goes through both of these pins  
# in this order is false.
```

Few recommendations on setting false paths are given below. To set a false path between two clock domains, use:

```
set_false_path -from [get_clocks clockA] \  
  -to [get_clocks clockB]
```

instead of:

```
set_false_path -from [get_pins {regA_*}/CP] \  
  -to [get_pins {regB_*}/D]
```

The second form is much slower.

Another recommendation is to minimize the usage of *-through* options, as it adds unnecessary runtime complexity. The *-through* option should only be used where it is absolutely necessary and there is no alternate way to specify the false path.

From an optimization perspective, another guideline is to not use a false path when a multicycle path is the real intent. If a signal is sampled at a known or predictable time, no matter how far out, a multicycle path specification should be used so that the path has some constraint and gets optimized to meet the multicycle constraint. If a false path is used on a path

that is sampled many clock cycles later, optimization of the remaining logic may invariably slow this path even beyond what may be necessary.

8.5 Half-Cycle Paths

If a design has both negative-edge triggered flip-flops (active clock edge is falling edge) and positive-edge triggered flip-flops (active clock edge is rising edge), it is likely that half-cycle paths exist in the design. A half-cycle path could be from a rising edge flip-flop to a falling edge flip-flop, or vice versa. Figure 8-19 shows an example when the launch is on the falling edge of the clock of flip-flop *UFF5*, and the capture is on the rising edge of the clock of flip-flop *UFF3*.

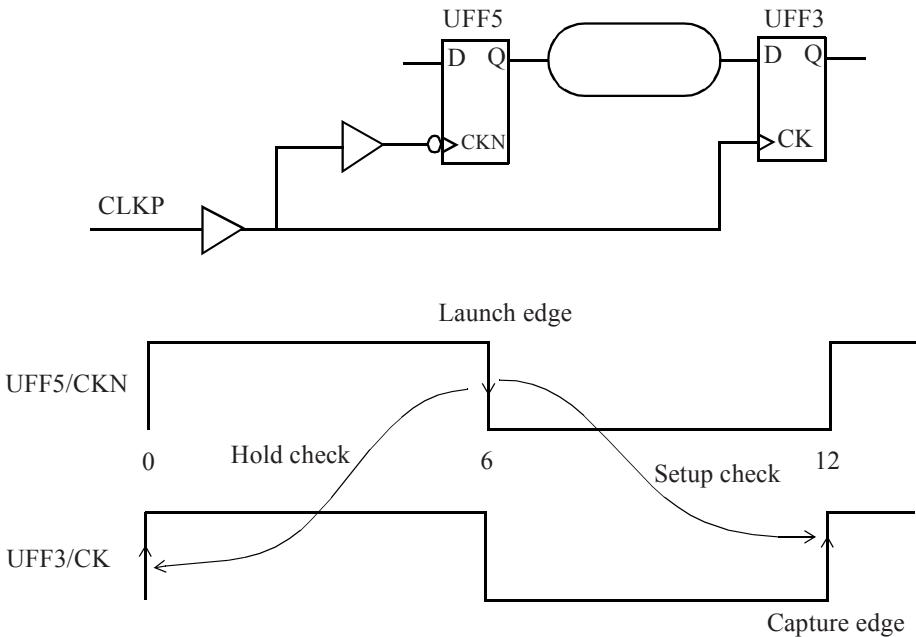


Figure 8-19 *A half-cycle path.*

Here is the setup timing check path report.

Startpoint: UFF5(**falling edge-triggered** flip-flop clocked by CLKP)
 Endpoint: UFF3 (**rising edge-triggered** flip-flop clocked by CLKP)
 Path Group: CLKP
 Path Type: **max**

Point	Incr	Path

clock CLKP (fall edge)	6.00	6.00
clock source latency	0.00	6.00
CLKP (in)	0.00	6.00 f
UCKBUF4/C (CKB)	0.06	6.06 f
UCKBUF6/C (CKB)	0.06	6.12 f
UFF5/CKN (DFN)	0.00	6.12 f
UFF5/Q (DFN) <-	0.16	6.28 r
UNAND0/ZN (ND2)	0.03	6.31 f
UFF3/D (DFF)	0.00	6.31 f
data arrival time		6.31
clock CLKP (rise edge)	12.00	12.00
clock source latency	0.00	12.00
CLKP (in)	0.00	12.00 r
UCKBUF4/C (CKB)	0.07	12.07 r
UFF3/CK (DFF)	0.00	12.07 r
clock uncertainty	-0.30	11.77
library setup time	-0.03	11.74
data required time		11.74

data required time		11.74
data arrival time		-6.31

slack (MET)		5.43

Note the edge specification in the *Startpoint* and *Endpoint*. The falling edge occurs at 6ns and the rising edge occurs at 12ns. Thus, the data gets only a half-cycle, which is 6ns, to propagate to the capture flip-flop.

While the data path gets only half-cycle for setup check, an extra half-cycle is available for the hold timing check. Here is the hold timing path.

Startpoint: UFF5 (**falling edge-triggered** flip-flop clocked by CLKP)
Endpoint: UFF3 (**rising edge-triggered** flip-flop clocked by CLKP)
Path Group: CLKP
Path Type: **min**

Point	Incr	Path

clock CLKP (fall edge)	6.00	6.00
clock source latency	0.00	6.00
CLKP (in)	0.00	6.00 f
UCKBUF4/C (CKB)	0.06	6.06 f
UCKBUF6/C (CKB)	0.06	6.12 f
UFF5/CKN (DFN)	0.00	6.12 f
UFF5/Q (DFN) <--	0.16	6.28 r
UNAND0/ZN (ND2)	0.03	6.31 f
UFF3/D (DFF)	0.00	6.31 f
data arrival time		6.31
clock CLKP (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKP (in)	0.00	0.00 r
UCKBUF4/C (CKB)	0.07	0.07 r
UFF3/CK (DFF)	0.00	0.07 r
clock uncertainty	0.05	0.12
library hold time	0.02	0.13
data required time		0.13

data required time		0.13
data arrival time		-6.31

slack (MET)		6.18

The hold check always occurs one cycle prior to the capture edge. Since the capture edge occurs at 12ns, the previous capture edge is at 0ns, and hence the hold gets checked at 0ns. This effectively adds a half-cycle margin for hold checking and thus results in a large positive slack on hold.

8.6 Removal Timing Check

A **removal timing check** ensures that there is adequate time between an active clock edge and the release of an asynchronous control signal. The check ensures that the active clock edge has no effect because the asynchronous control signal remains active until *removal time* after the active clock edge. In other words, the asynchronous control signal is released (becomes inactive) well after the active clock edge so that the clock edge can have no effect. This is illustrated in Figure 8-20. This check is based on the removal time specified for the asynchronous pin of the flip-flop. Here is an excerpt from the cell library description corresponding to the removal check.

```
pin(CDN) {
    . . .
    timing() {
        related_pin : "CK";
        timing_type : removal_rising;
        . . .
    }
}
```

Like a hold check, it is a min path check except that it is on an asynchronous pin of a flip-flop.

Startpoint: UFF5(falling edge-triggered flip-flop clocked by CLKP)
 Endpoint: UFF6 (**removal check** against rising-edge clock CLKP)
 Path Group: ****async_default****
 Path Type: min

Point	Incr	Path
-----	-----	-----
clock CLKP (fall edge)	6.00	6.00
clock source latency	0.00	6.00
CLKP (in)	0.00	6.00 f
UCKBUF4/C (CKB)	0.06	6.06 f
UCKBUF6/C (CKB)	0.07	6.13 f
UFF5/CKN (DFN)	0.00	6.13 f

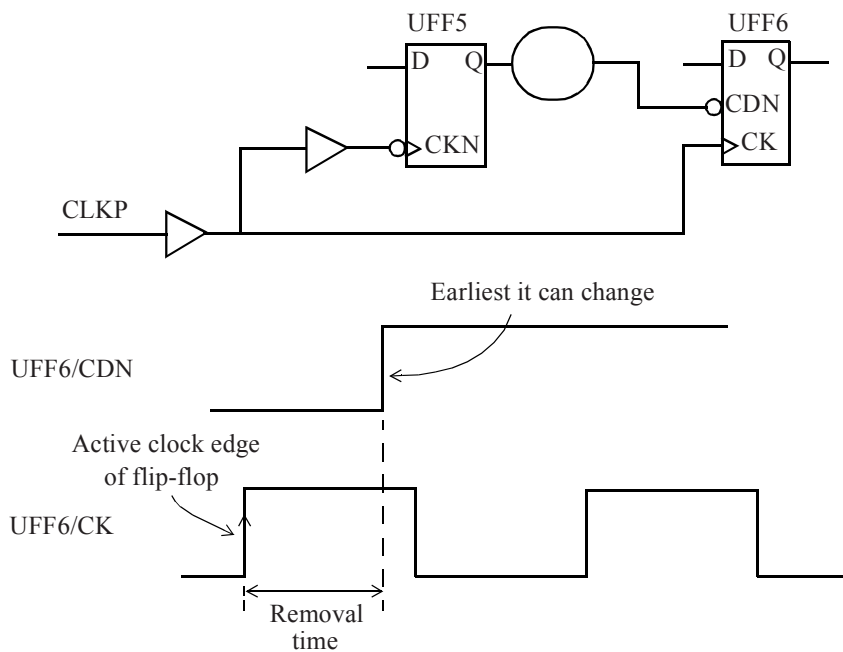


Figure 8-20 *Removal timing check.*

UFF5/Q (DFN)	0.15	6.28 f
UINV8/ZN (INV)	0.03	6.31 r
UFF6/CDN (DFCN)	0.00	6.31 r
data arrival time		6.31
clock CLKP (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKP (in)	0.00	0.00 r
UCKBUF4/C (CKB)	0.07	0.07 r
UCKBUF6/C (CKB)	0.07	0.14 r
UCKBUF7/C (CKB)	0.05	0.19 r
UFF6/CK (DFCN)	0.00	0.19 r
clock uncertainty	0.05	0.24
library removal time	0.19	0.43
data required time		0.43

data required time	0.43
data arrival time	-6.31

slack (MET)	5.88

The *Endpoint* shows that it is removal check. It is on the asynchronous pin *CDN* of flip-flop *UFF6*. The removal time for this flip-flop is listed as *library removal time* with a value of 0.19ns.

All asynchronous timing checks are assigned to the *async default* path group.

8.7 Recovery Timing Check

A **recovery timing check** ensures that there is a minimum amount of time between the asynchronous signal becoming inactive and the next active clock edge. In other words, this check ensures that after the asynchronous signal becomes inactive, there is adequate time to recover so that the next active clock edge can be effective. For example, consider the time between an asynchronous reset becoming inactive and the clock active edge of a flip-flop. If the active clock edge occurs too soon after the release of reset, the state of the flip-flop may be unknown. The recovery check is illustrated in Figure 8-21. This check is based upon the recovery time specified for the asynchronous pin of the flip-flop in its cell library file, an excerpt of which is shown below.

```
pin(RSN) {
    . . .
    timing() {
        related_pin : "CK";
        timing_type : recovery_rising;
        . . .
    }
}
```

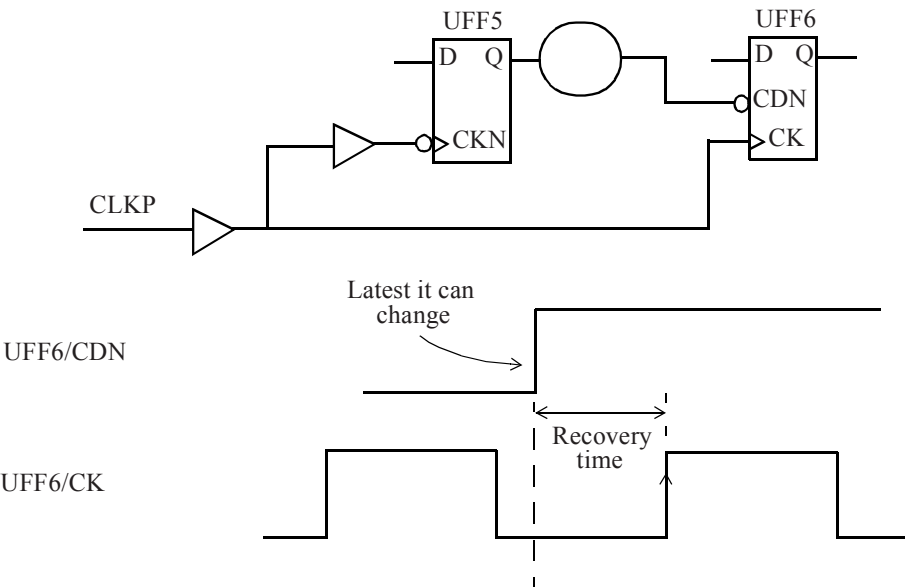


Figure 8-21 *Recovery timing check.*

Like a setup check, this is a max path check except that it is on an asynchronous signal.

Here is a recovery path report.

Startpoint: UFF5(falling edge-triggered flip-flop clocked by CLKP)
Endpoint: UFF6 (**recovery check** against rising-edge clock CLKP)
Path Group: ****async_default****
Path Type: max

Point	Incr	Path

clock CLKP (fall edge)	6.00	6.00
clock source latency	0.00	6.00
CLKP (in)	0.00	6.00 f

UCKBUF4/C (CKB)	0.06	6.06 f
UCKBUF6/C (CKB)	0.07	6.13 f
UFF5/CKN (DFN)	0.00	6.13 f
UFF5/Q (DFN)	0.15	6.28 f
UINV8/ZN (INV)	0.03	6.31 r
UFF6/CDN (DFCN)	0.00	6.31 r
data arrival time		6.31
clock CLKP (rise edge)	12.00	12.00
clock source latency	0.00	12.00
CLKP (in)	0.00	12.00 r
UCKBUF4/C (CKB)	0.07	12.07 r
UCKBUF6/C (CKB)	0.07	12.14 r
UCKBUF7/C (CKB)	0.05	12.19 r
UFF6/CK (DFCN)	0.00	12.19 r
clock uncertainty	-0.30	11.89
library recovery time	0.09	11.98
data required time		11.98

data required time		11.98
data arrival time		-6.31

slack (MET)		5.67

The *Endpoint* shows that it is recovery check. The recovery time for the *UFF6* flip-flop is listed as the *library recovery time* with a value of 0.09ns. Recovery checks also belong to the *async default* path group.

8.8 Timing across Clock Domains

8.8.1 Slow to Fast Clock Domains

Let us examine the setup and hold checks when a path goes from a slower clock domain to a faster clock domain. This is shown in Figure 8-22.

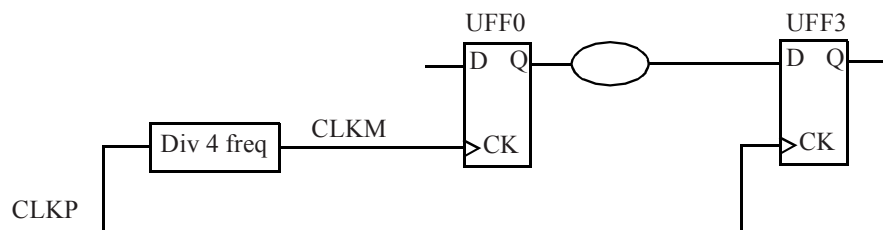


Figure 8-22 *Path from a slow clock to a faster clock.*

Here are the clock definitions for our example.

```
create_clock -name CLKM \
  -period 20 -waveform {0 10} [get_ports CLKM]
create_clock -name CLKP \
  -period 5 -waveform {0 2.5} [get_ports CLKP]
```

When the clock frequencies are different for the launch flip-flop and the capture flip-flop, STA is performed by first determining a common base period. An example of a message produced when STA is performed on such a design with the above two clocks is given below. The faster clock is expanded so that a common period is obtained.

```
Expanding clock 'CLKP' to base period of 20.00
(old period was 5.00, added 6 edges).
```

Figure 8-23 shows the setup check. By default, the most constraining setup edge relationship is used, which in this case is the very next capture edge. Here is a setup path report that shows this.

```
Startpoint: UFF0 (rising edge-triggered flip-flop clocked by CLKM)
Endpoint: UFF3 (rising edge-triggered flip-flop clocked by CLKP)
Path Group: CLKP
Path Type: max
```

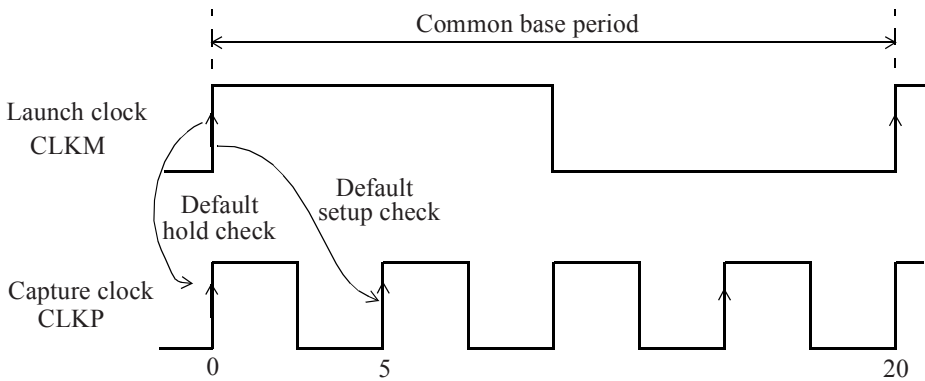



Figure 8-23 Setup and hold checks with slow to fast path.

Point	Incr	Path

clock CLKM (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKM (in)	0.00	0.00 r
UCKBUF0/C (CKB)	0.06	0.06 r
UCKBUF1/C (CKB)	0.06	0.11 r
UFF0/CK (DFF)	0.00	0.11 r
UFF0/Q (DFF) <=	0.14	0.26 f
UNAND0/ZN (ND2)	0.03	0.29 r
UFF3/D (DFF)	0.00	0.29 r
data arrival time		0.29
clock CLKP (rise edge)	5.00	5.00
clock source latency	0.00	5.00
CLKP (in)	0.00	5.00 r
UCKBUF4/C (CKB)	0.07	5.07 r
UFF3/CK (DFF)	0.00	5.07 r
clock uncertainty	-0.30	4.77
library setup time	-0.04	4.72
data required time		4.72

data required time		4.72
data arrival time		-0.29

slack (MET)		4.44

Notice that the launch clock is at time 0ns while the capture clock is at time 5ns.

As discussed earlier, hold checks are related to the setup checks and ensure that the data launched by a clock edge does not interfere with the previous capture. Here is the hold check timing report.

Startpoint: UFF0 (rising edge-triggered flip-flop clocked by CLKM)
Endpoint: UFF3 (rising edge-triggered flip-flop clocked by CLKP)
Path Group: CLKP
Path Type: **min**

Point	Incr	Path

clock CLKM (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKM (in)	0.00	0.00 r
UCKBUF0/C (CKB)	0.06	0.06 r
UCKBUF1/C (CKB)	0.06	0.11 r
UFF0/CK (DFF)	0.00	0.11 r
UFF0/Q (DFF) <-	0.14	0.26 r
UNAND0/ZN (ND2)	0.03	0.29 f
UFF3/D (DFF)	0.00	0.29 f
data arrival time		0.29
clock CLKP (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKP (in)	0.00	0.00 r
UCKBUF4/C (CKB)	0.07	0.07 r
UFF3/CK (DFF)	0.00	0.07 r
clock uncertainty	0.05	0.12
library hold time	0.02	0.13
data required time		0.13

data required time		0.13
data arrival time		-0.29

slack (MET)		0.16

In the above example, we can see that the launch data is available every fourth cycle of the capture clock. Let us assume that the intention is not to capture data on the very next active edge of *CLKP*, but to capture on every 4th capture edge. This assumption gives the combinational logic between the flip-flops four periods of *CLKP* to propagate, which is 20ns. We can do this by setting the following multicycle specification:

```
set_multicycle_path 4 -setup \
  -from [get_clocks CLKM] -to [get_clocks CLKP] -end
```

The *-end* specifies that the multicycle of 4 refers to the end point or the capture clock. This multicycle specification changes the setup and hold checks to the ones shown in Figure 8-24. Here is the setup report.

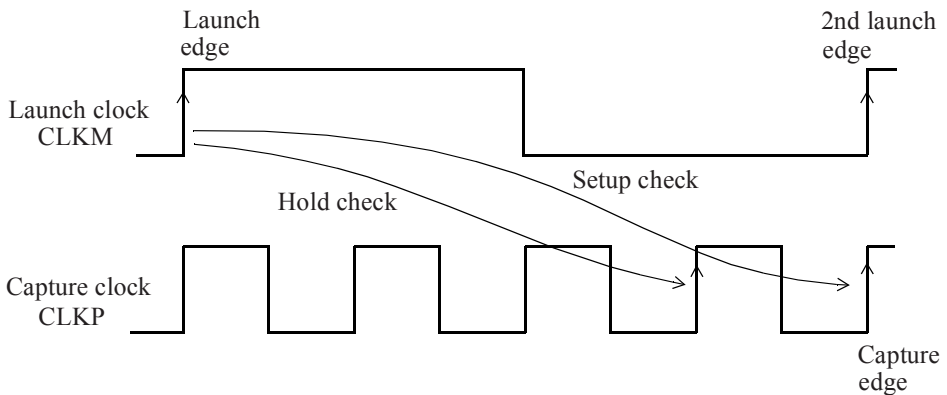


Figure 8-24 *Multicycle of 4 between clock domains.*

```
Startpoint: UFF0 (rising edge-triggered flip-flop clocked by CLKM)
Endpoint: UFF3 (rising edge-triggered flip-flop clocked by CLKP)
Path Group: CLKP
Path Type: max
```

Point	Incr	Path

clock CLKM (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKM (in)	0.00	0.00 r
UCKBUF0/C (CKB)	0.06	0.06 r
UCKBUF1/C (CKB)	0.06	0.11 r
UFF0/CK (DFF)	0.00	0.11 r
UFF0/Q (DFF) <-	0.14	0.26 f
UNAND0/ZN (ND2)	0.03	0.29 r
UFF3/D (DFF)	0.00	0.29 r
data arrival time		0.29
clock CLKP (rise edge)	20.00	20.00
clock source latency	0.00	20.00
CLKP (in)	0.00	20.00 r
UCKBUF4/C (CKB)	0.07	20.07 r
UFF3/CK (DFF)	0.00	20.07 r
clock uncertainty	-0.30	19.77
library setup time	-0.04	19.72
data required time		19.72

data required time		19.72
data arrival time		-0.29

slack (MET)		19.44

Figure 8-24 shows the hold check - note that the hold check is derived from the setup check and defaults to one cycle preceding the intended capture edge. Here is the hold timing report. Notice that the hold capture edge is at 15ns, one cycle prior to the setup capture edge.

Startpoint: UFF0 (rising edge-triggered flip-flop clocked by CLKM)
Endpoint: UFF3 (rising edge-triggered flip-flop clocked by CLKP)
Path Group: CLKP
Path Type: **min**

Point	Incr	Path

clock CLKM (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKM (in)	0.00	0.00 r

UCKBUF0/C (CKB)	0.06	0.06 r
UCKBUF1/C (CKB)	0.06	0.11 r
UFF0/CK (DFF)	0.00	0.11 r
UFF0/Q (DFF) <-	0.14	0.26 r
UNAND0/ZN (ND2)	0.03	0.29 f
UFF3/D (DFF)	0.00	0.29 f
data arrival time		0.29
clock CLKP (rise edge)	15.00	15.00
clock source latency	0.00	15.00
CLKP (in)	0.00	15.00 r
UCKBUF4/C (CKB)	0.07	15.07 r
UFF3/CK (DFF)	0.00	15.07 r
clock uncertainty	0.05	15.12
library hold time	0.02	15.13
data required time		15.13

data required time		15.13
data arrival time		-0.29

slack (VIOLATED)		-14.84

In most designs, this is not the intended check, and the hold check should be moved all the way back to where the launch edge is. We do this by setting a hold multicycle specification of 3.

```
set_multicycle_path 3 -hold \
  -from [get_clocks CLKM] -to [get_clocks CLKP] -end
```

The cycle of 3 moves the hold checking edge *back* three cycles, that is, to time 0ns. The distinction with a setup multicycle is that in setup, the setup capture edge moves forward by the specified number of cycles from the default setup capture edge; in a hold multicycle, the hold check edge moves backward from the default hold check edge (one cycle before setup edge). The *-end* option implies that we want to move the endpoint (or capture edge) back by the specified number of cycles, which is that of the capture clock. Instead of *-end*, the other choice, the *-start* option, specifies the number of launch clock cycles to move by; the *-end* option specifies the

number of capture clock cycles to move by. The *-end* is the default for a multicycle setup and the *-start* is the default for multicycle hold.

With the additional multicycle hold specification, the clock edge used for hold timing checks is moved back one cycle, and the checks look like those shown in Figure 8-25. The hold report with the multicycle hold specification is as follows.

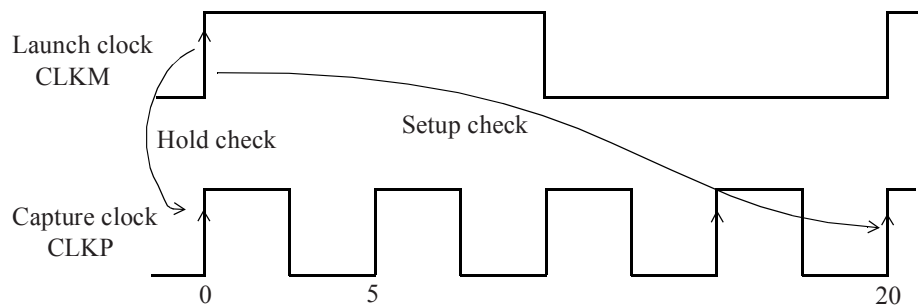


Figure 8-25 *Hold time relaxed with multicycle hold specification.*

Startpoint: UFF0 (rising edge-triggered flip-flop clocked by CLKM)
 Endpoint: UFF3 (rising edge-triggered flip-flop clocked by CLKP)
 Path Group: CLKP
 Path Type: **min**

Point	Incr	Path

clock CLKM (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKM (in)	0.00	0.00 r
UCKBUF0/C (CKB)	0.06	0.06 r
UCKBUF1/C (CKB)	0.06	0.11 r
UFF0/CK (DF)	0.00	0.11 r
UFF0/Q (DF) <=	0.14	0.26 r
UNAND0/ZN (ND2)	0.03	0.29 f
UFF3/D (DF)	0.00	0.29 f
data arrival time		0.29

clock CLKP (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKP (in)	0.00	0.00 r
UCKBUF4/C (CKB)	0.07	0.07 r
UFF3/CK (DF)	0.00	0.07 r
clock uncertainty	0.05	0.12
library hold time	0.02	0.13
data required time		0.13

data required time		0.13
data arrival time		-0.29

slack (MET)		0.16

In summary, if a setup multicycle of N cycles is specified, then most likely a hold multicycle of $N-1$ cycles should also be specified. A good rule of thumb for multi-frequency multicycle path specification in the case of paths between slow to fast clock domains is to use the *-end* option. With this option, the setup and hold checks are adjusted based upon the clock cycles of the fast clock.

8.8.2 Fast to Slow Clock Domains

In this subsection, we consider examples where the data path goes from a fast clock domain to a slow clock domain. The default setup and hold checks are as shown in Figure 8-26 when the following clock definitions are used.

```
create_clock -name CLKM \
  -period 20 -waveform {0 10} [get_ports CLKM]
create_clock -name CLKP \
  -period 5 -waveform {0 2.5} [get_ports CLKP]
```

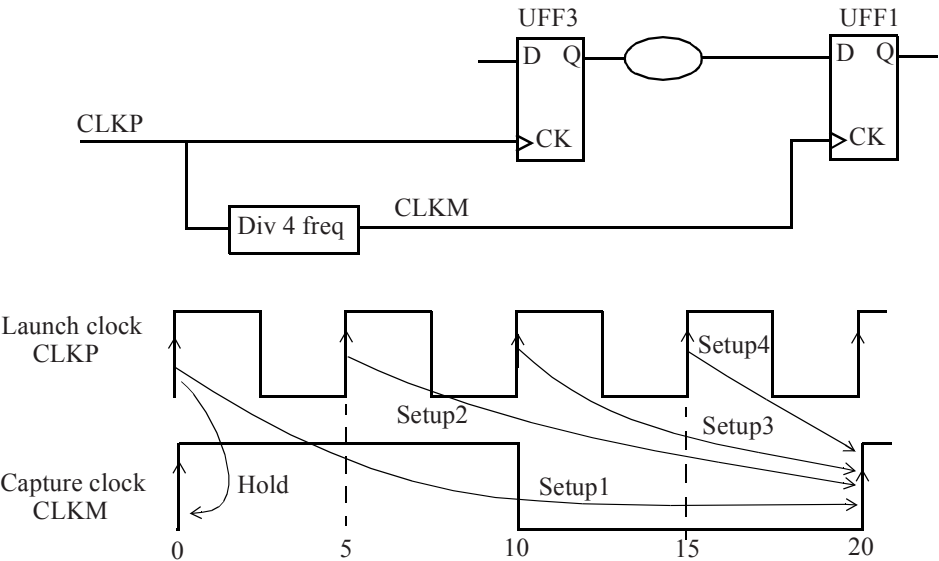


Figure 8-26 Path from fast clock to slow clock domain.

There are four setup timing checks possible; see *Setup1*, *Setup2*, *Setup3* and *Setup4* in the figure. However, the most restrictive one is the *Setup4* check. Here is the path report of this most restrictive path. Notice that the launch clock edge is at 15ns and the capture clock edge is at 20ns.

Startpoint: UFF3 (rising edge-triggered flip-flop clocked by **CLKP**)
Endpoint: UFF1 (rising edge-triggered flip-flop clocked by **CLKM**)
Path Group: **CLKM**
Path Type: **max**

Point	Incr	Path

clock CLKP (rise edge)	15.00	15.00
clock source latency	0.00	15.00
CLKP (in)	0.00	15.00 r
UCKBUF4/C (CKB)	0.07	15.07 r
UFF3/CK (DFF)	0.00	15.07 r

UFF3/Q (DFF) <-	0.15	15.22 f
UNOR0/ZN (NR2)	0.05	15.27 r
UBUF4/Z (BUFF)	0.05	15.32 r
UFF1/D (DFF)	0.00	15.32 r
data arrival time		15.32
clock CLKM (rise edge)	20.00	20.00
clock source latency	0.00	20.00
CLKM (in)	0.00	20.00 r
UCKBUF0/C (CKB)	0.06	20.06 r
UCKBUF2/C (CKB)	0.07	20.12 r
UFF1/CK (DFF)	0.00	20.12 r
clock uncertainty	-0.30	19.82
library setup time	-0.04	19.78
data required time		19.78

data required time		19.78
data arrival time		-15.32

slack (MET)		4.46

Similar to the setup checks, there are four hold checks possible. Figure 8-26 shows the most restrictive hold check which ensures that the capture edge at 0ns does not capture the data being launched at 0ns. Here is the timing report for this hold check.

Startpoint: UFF3 (rising edge-triggered flip-flop clocked by **CLKP**)
 Endpoint: UFF1 (rising edge-triggered flip-flop clocked by **CLKM**)
 Path Group: **CLKM**
 Path Type: **min**

Point	Incr	Path

clock CLKP (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKP (in)	0.00	0.00 r
UCKBUF4/C (CKB)	0.07	0.07 r
UFF3/CK (DFF)	0.00	0.07 r
UFF3/Q (DFF) <-	0.16	0.22 r
UNOR0/ZN (NR2)	0.02	0.25 f

UBUF4/Z (BUFF)	0.06	0.30 f
UFF1/D (DFF)	0.00	0.30 f
data arrival time		0.30
clock CLKM (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKM (in)	0.00	0.00 r
UCKBUF0/C (CKB)	0.06	0.06 r
UCKBUF2/C (CKB)	0.07	0.12 r
UFF1/CK (DFF)	0.00	0.12 r
clock uncertainty	0.05	0.17
library hold time	0.01	0.19
data required time		0.19

data required time		0.19
data arrival time		-0.30

slack (MET)		0.12

In general, a designer may specify the data path from the fast clock to the slow clock to be a multicycle path. If the setup check is relaxed to provide two cycles of the faster clock for the data path, the following is included for this multicycle specification:

```
set_multicycle_path 2 -setup \  
  -from [get_clocks CLKP] -to [get_clocks CLKM] -start  
  
set_multicycle_path 1 -hold \  
  -from [get_clocks CLKP] -to [get_clocks CLKM] -start  
# The -start option refers to the launch clock and is  
# the default for a multicycle hold.
```

In this case, Figure 8-27 shows the clock edges used for the setup and hold checks. The *-start* option specifies that the unit for the number of cycles (2 in this case) is that of the launch clock (*CLKP* in this case). The setup multicycle of 2 moves the launch edge one edge prior to the default launch edge, that is, at 10ns instead of the default 15ns. The hold multicycle ensures that

the capture of the earlier data can reliably occur at 0ns due to the launch edge also at 0ns.

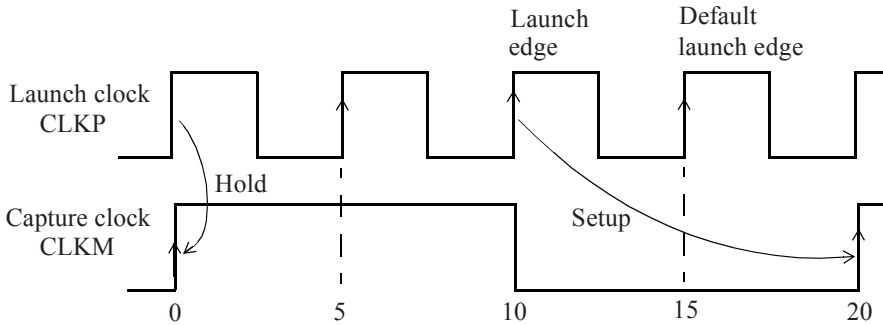


Figure 8-27 Setup multicycle of 2.

Here is the setup path report. As expected, the launch clock edge is at 10ns and the capture clock edge is at 20ns.

Startpoint: UFF3 (rising edge-triggered flip-flop clocked by **CLKP**)
 Endpoint: UFF1 (rising edge-triggered flip-flop clocked by **CLKM**)
 Path Group: **CLKM**
 Path Type: **max**

Point	Incr	Path

clock CLKP (rise edge)	10.00	10.00
clock source latency	0.00	10.00
CLKP (in)	0.00	10.00 r
UCKBUF4/C (CKB)	0.07	10.07 r
UFF3/CK (DFF)	0.00	10.07 r
UFF3/Q (DFF) <--	0.15	10.22 f
UNOR0/ZN (NR2)	0.05	10.27 r
UBUF4/Z (BUFF)	0.05	10.32 r
UFF1/D (DFF)	0.00	10.32 r
data arrival time		10.32

clock CLKM (rise edge)	20.00	20.00
clock source latency	0.00	20.00
CLKM (in)	0.00	20.00 r
UCKBUF0/C (CKB)	0.06	20.06 r
UCKBUF2/C (CKB)	0.07	20.12 r
UFF1/CK (DFF)	0.00	20.12 r
clock uncertainty	-0.30	19.82
library setup time	-0.04	19.78
data required time		19.78

data required time		19.78
data arrival time		-10.32

slack (MET)		9.46

Here is the hold path timing report. The hold check is at 0ns where both the capture and launch clocks have rising edges.

Startpoint: UFF3 (rising edge-triggered flip-flop clocked by **CLKP**)
Endpoint: UFF1 (rising edge-triggered flip-flop clocked by **CLKM**)
Path Group: **CLKM**
Path Type: **min**

Point	Incr	Path

clock CLKP (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKP (in)	0.00	0.00 r
UCKBUF4/C (CKB)	0.07	0.07 r
UFF3/CK (DFF)	0.00	0.07 r
UFF3/Q (DFF) <-	0.16	0.22 r
UNOR0/ZN (NR2)	0.02	0.25 f
UBUF4/Z (BUFF)	0.06	0.30 f
UFF1/D (DFF)	0.00	0.30 f
data arrival time		0.30
clock CLKM (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKM (in)	0.00	0.00 r
UCKBUF0/C (CKB)	0.06	0.06 r
UCKBUF2/C (CKB)	0.07	0.12 r

	Examples	SECTION 8.9
UFF1/CK (DFF)	0.00	0.12 r
clock uncertainty	0.05	0.17
library hold time	0.01	0.19
data required time		0.19

data required time		0.19
data arrival time		-0.30

slack (MET)		0.12

Unlike the case of paths from slow to fast clock domains, a good rule of thumb for multi-frequency multicycle path specification in the case of paths from fast to slow clock domains is to use the *-start* option. The setup and hold checks are then adjusted based upon the fast clock.

8.9 Examples

In this section, we describe different scenarios of launch and capture clocks and show how the setup and hold checks are performed. Figure 8-28 shows the configuration for the examples.

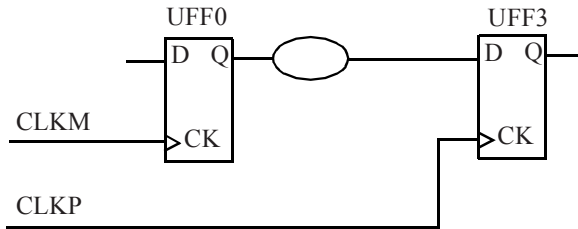


Figure 8-28 *Launch and capture clocks with different relationships.*

Half-cycle Path - Case 1

In this example, the two clocks have the same period but are of opposite phase. Here are the clock specifications - the waveforms are shown in Figure 8-29.

```
create_clock -name CLKM \
  -period 20 -waveform {0 10} [get_ports CLKM]
create_clock -name CLKP \
  -period 20 -waveform {10 20} [get_ports CLKP]
```

The setup check is from a launch edge, at 0ns, to the next capture edge, at 10ns. A half-cycle margin is available for the hold check which validates whether the data launched at 20ns is not captured by the capture edge at 10ns. Here is the setup path report.

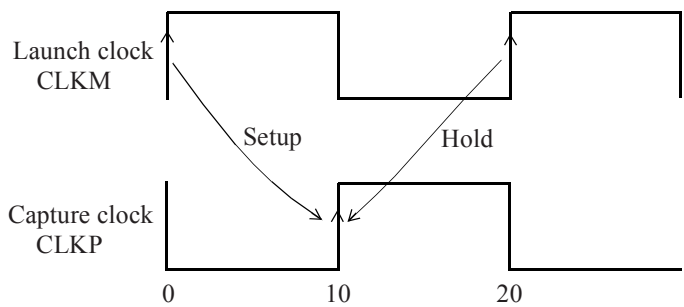


Figure 8-29 *Clock waveforms for half-cycle path - case 1.*

```
Startpoint: UFF0 (rising edge-triggered flip-flop clocked by CLKM)
Endpoint: UFF3 (rising edge-triggered flip-flop clocked by CLKP)
Path Group: CLKP
Path Type: max
```

Point	Incr	Path

clock CLKM (rise edge)	0.00	0.00

	Examples	SECTION 8.9
clock source latency	0.00	0.00
CLKM (in)	0.00	0.00 r
UCKBUF0/C (CKB)	0.06	0.06 r
UCKBUF1/C (CKB)	0.06	0.11 r
UFF0/CK (DFF)	0.00	0.11 r
UFF0/Q (DFF) <--	0.14	0.26 f
UNAND0/ZN (ND2)	0.03	0.29 r
UFF3/D (DFF)	0.00	0.29 r
data arrival time		0.29
clock CLKP (rise edge)	10.00	10.00
clock source latency	0.00	10.00
CLKP (in)	0.00	10.00 r
UCKBUF4/C (CKB)	0.07	10.07 r
UFF3/CK (DFF)	0.00	10.07 r
clock uncertainty	-0.30	9.77
library setup time	-0.04	9.72
data required time		9.72

data required time		9.72
data arrival time		-0.29

slack (MET)		9.44

Here is the hold path report.

Startpoint: UFF0 (rising edge-triggered flip-flop clocked by **CLKM**)
 Endpoint: UFF3 (rising edge-triggered flip-flop clocked by **CLKP**)
 Path Group: **CLKP**
 Path Type: **min**

Point	Incr	Path

clock CLKM (rise edge)	20.00	20.00
clock source latency	0.00	20.00
CLKM (in)	0.00	20.00 r
UCKBUF0/C (CKB)	0.06	20.06 r
UCKBUF1/C (CKB)	0.06	20.11 r
UFF0/CK (DFF)	0.00	20.11 r
UFF0/Q (DFF) <--	0.14	20.26 r
UNAND0/ZN (ND2)	0.03	20.29 f

UFF3/D (DFF)	0.00	20.29 f
data arrival time		20.29
clock CLKP (rise edge)	10.00	10.00
clock source latency	0.00	10.00
CLKP (in)	0.00	10.00 r
UCKBUF4/C (CKB)	0.07	10.07 r
UFF3/CK (DFF)	0.00	10.07 r
clock uncertainty	0.05	10.12
library hold time	0.02	10.13
data required time		10.13

data required time		10.13
data arrival time		-20.29

slack (MET)		10.16

Half-cycle Path - Case 2

This example is similar to case 1 and the launch and capture clocks are of opposite phase. The launch clock is shifted in time. Here are the clock specifications; the waveforms are shown in Figure 8-30.

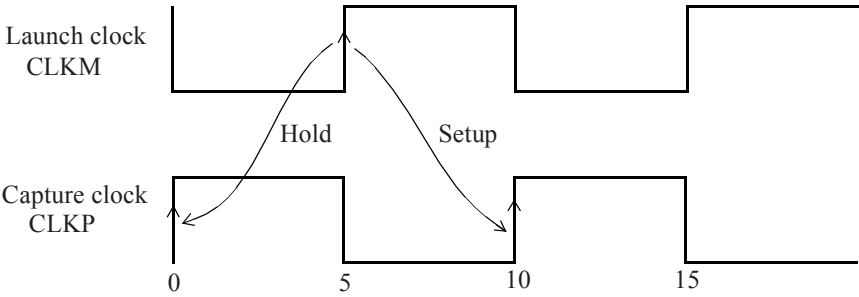


Figure 8-30 *Clock waveforms for half-cycle path - case 2.*

```
create_clock -name CLKM \  
-period 10 -waveform {5 10} [get_ports CLKM]
```



```
create_clock -name CLKP \
-period 10 -waveform {0 5} [get_ports CLKP]
```

The setup check is from the launch clock edge at 5ns to the next capture clock edge at 10ns. The hold check is from the launch edge at 5ns to the capture edge at 0ns. Here is the setup path report.

Startpoint: UFF0 (rising edge-triggered flip-flop clocked by **CLKM**)
 Endpoint: UFF3 (rising edge-triggered flip-flop clocked by **CLKP**)
 Path Group: **CLKP**
 Path Type: **max**

Point	Incr	Path

clock CLKM (rise edge)	5.00	5.00
clock source latency	0.00	5.00
CLKM (in)	0.00	5.00 r
UCKBUF0/C (CKB)	0.06	5.06 r
UCKBUF1/C (CKB)	0.06	5.11 r
UFF0/CK (DFF)	0.00	5.11 r
UFF0/Q (DFF) <-	0.14	5.26 f
UNAND0/ZN (ND2)	0.03	5.29 r
UFF3/D (DFF)	0.00	5.29 r
data arrival time		5.29
clock CLKP (rise edge)	10.00	10.00
clock source latency	0.00	10.00
CLKP (in)	0.00	10.00 r
UCKBUF4/C (CKB)	0.07	10.07 r
UFF3/CK (DFF)	0.00	10.07 r
clock uncertainty	-0.30	9.77
library setup time	-0.04	9.72
data required time		9.72

data required time		9.72
data arrival time		-5.29

slack (MET)		4.44

Here is the hold path timing report.

Startpoint: UFF0 (rising edge-triggered flip-flop clocked by **CLKM**)
Endpoint: UFF3 (rising edge-triggered flip-flop clocked by **CLKP**)
Path Group: **CLKP**
Path Type: **min**

Point	Incr	Path

clock CLKM (rise edge)	5.00	5.00
clock source latency	0.00	5.00
CLKM (in)	0.00	5.00 r
UCKBUF0/C (CKB)	0.06	5.06 r
UCKBUF1/C (CKB)	0.06	5.11 r
UFF0/CK (DFF)	0.00	5.11 r
UFF0/Q (DFF) <-	0.14	5.26 r
UNAND0/ZN (ND2)	0.03	5.29 f
UFF3/D (DFF)	0.00	5.29 f
data arrival time		5.29
clock CLKP (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKP (in)	0.00	0.00 r
UCKBUF4/C (CKB)	0.07	0.07 r
UFF3/CK (DFF)	0.00	0.07 r
clock uncertainty	0.05	0.12
library hold time	0.02	0.13
data required time		0.13

data required time		0.13
data arrival time		-5.29

slack (MET)		5.16

Fast to Slow Clock Domain

In this example, the capture clock is a divide-by-2 of the launch clock. Here are the clock specifications.

```
create_clock -name CLKM \  
-period 10 -waveform {0 5} [get_ports CLKM]  
create_clock -name CLKP \  
-period 20 -waveform {0 10} [get_ports CLKP]
```

The waveforms are shown in Figure 8-31. The setup check is from the launch edge at 10ns to the capture edge at 20ns. The hold check is from the launch edge at 0ns to the capture edge at 0ns. Here is the setup path report.

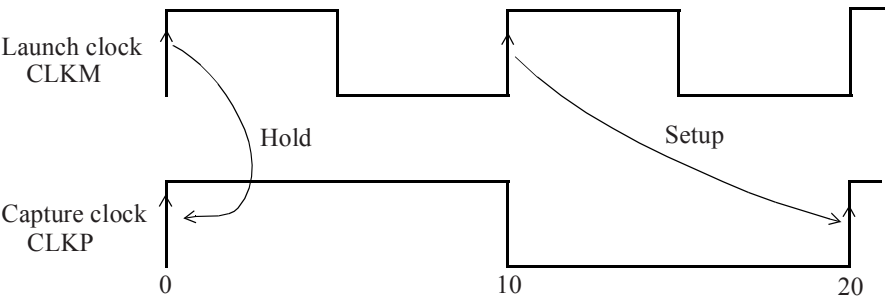


Figure 8-31 *Fast to slow clock domain example clocks.*

Startpoint: UFF0 (rising edge-triggered flip-flop clocked by **CLKM**)
Endpoint: UFF3 (rising edge-triggered flip-flop clocked by **CLKP**)
Path Group: **CLKP**
Path Type: **max**

Point	Incr	Path

clock CLKM (rise edge)	10.00	10.00
clock source latency	0.00	10.00

CLKM (in)	0.00	10.00 r
UCKBUF0/C (CKB)	0.06	10.06 r
UCKBUF1/C (CKB)	0.06	10.11 r
UFF0/CK (DFF)	0.00	10.11 r
UFF0/Q (DFF) <=	0.14	10.26 f
UNAND0/ZN (ND2)	0.03	10.29 r
UFF3/D (DFF)	0.00	10.29 r
data arrival time		10.29
clock CLKP (rise edge)	20.00	20.00
clock source latency	0.00	20.00
CLKP (in)	0.00	20.00 r
UCKBUF4/C (CKB)	0.07	20.07 r
UFF3/CK (DFF)	0.00	20.07 r
clock uncertainty	-0.30	19.77
library setup time	-0.04	19.72
data required time		19.72

data required time		19.72
data arrival time		-10.29

slack (MET)		9.44

Here is the hold path timing report.

Startpoint: UFF0 (rising edge-triggered flip-flop clocked by **CLKM**)
Endpoint: UFF3 (rising edge-triggered flip-flop clocked by **CLKP**)
Path Group: **CLKP**
Path Type: **min**

Point	Incr	Path

clock CLKM (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKM (in)	0.00	0.00 r
UCKBUF0/C (CKB)	0.06	0.06 r
UCKBUF1/C (CKB)	0.06	0.11 r
UFF0/CK (DFF)	0.00	0.11 r
UFF0/Q (DFF) <=	0.14	0.26 r
UNAND0/ZN (ND2)	0.03	0.29 f
UFF3/D (DFF)	0.00	0.29 f

data arrival time		0.29
clock CLKP (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKP (in)	0.00	0.00 r
UCKBUF4/C (CKB)	0.07	0.07 r
UFF3/CK (DFF)	0.00	0.07 r
clock uncertainty	0.05	0.12
library hold time	0.02	0.13
data required time		0.13

data required time		0.13
data arrival time		-0.29

slack (MET)		0.16

Slow to Fast Clock Domain

In this example, the capture clock is at 2x the speed of the launch clock. Figure 8-32 shows the clock edge corresponding to the setup and hold checks. Setup check is done from the launch edge, at 0ns, to the next capture edge, at 5ns. The hold check is done with the capture edge one cycle prior to the setup capture edge, that is, both launch and capture edges are at 0ns.

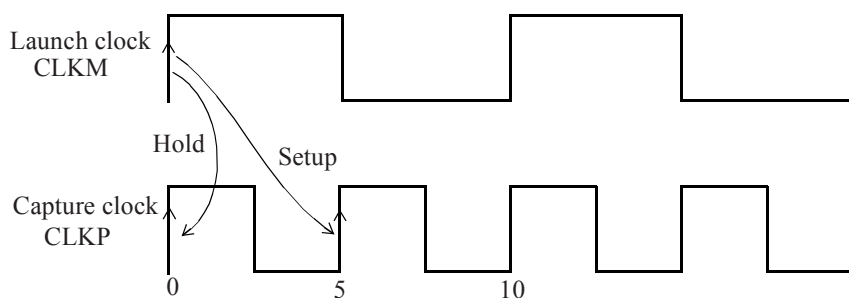


Figure 8-32 *Slow to fast clock domain example clocks.*

Here is the setup path report.

Startpoint: UFF0 (rising edge-triggered flip-flop clocked by **CLKM**)
Endpoint: UFF3 (rising edge-triggered flip-flop clocked by **CLKP**)
Path Group: **CLKP**
Path Type: **max**

Point	Incr	Path

clock CLKM (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKM (in)	0.00	0.00 r
UCKBUF0/C (CKB)	0.06	0.06 r
UCKBUF1/C (CKB)	0.06	0.11 r
UFF0/CK (DFF)	0.00	0.11 r
UFF0/Q (DFF) <-	0.14	0.26 f
UNAND0/ZN (ND2)	0.03	0.29 r
UFF3/D (DFF)	0.00	0.29 r
data arrival time		0.29
clock CLKP (rise edge)	5.00	5.00
clock source latency	0.00	5.00
CLKP (in)	0.00	5.00 r
UCKBUF4/C (CKB)	0.07	5.07 r
UFF3/CK (DFF)	0.00	5.07 r
clock uncertainty	-0.30	4.77
library setup time	-0.04	4.72
data required time		4.72

data required time		4.72
data arrival time		-0.29

slack (MET)		4.44

Here is the hold timing report.

Startpoint: UFF0 (rising edge-triggered flip-flop clocked by **CLKM**)
Endpoint: UFF3 (rising edge-triggered flip-flop clocked by **CLKP**)
Path Group: **CLKP**
Path Type: **min**

Point	Incr	Path

clock CLKM (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKM (in)	0.00	0.00 r
UCKBUF0/C (CKB)	0.06	0.06 r
UCKBUF1/C (CKB)	0.06	0.11 r
UFF0/CK (DFF)	0.00	0.11 r
UFF0/Q (DFF) <--	0.14	0.26 r
UNAND0/ZN (ND2)	0.03	0.29 f
UFF3/D (DFF)	0.00	0.29 f
data arrival time		0.29
clock CLKP (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKP (in)	0.00	0.00 r
UCKBUF4/C (CKB)	0.07	0.07 r
UFF3/CK (DFF)	0.00	0.07 r
clock uncertainty	0.05	0.12
library hold time	0.02	0.13
data required time		0.13

data required time		0.13
data arrival time		-0.29

slack (MET)		0.16

8.10 Multiple Clocks

8.10.1 Integer Multiples

Often there are multiple clocks defined in a design with frequencies that are simple (or integer) multiples of each other. In such cases, STA is performed by computing a common base period among all related clocks (two clocks are **related** if they have a data path between their domains). The common base period is established so that all clocks are synchronized.

Here is an example that shows four related clocks:

```
create_clock -name CLKM \
  -period 20 -waveform {0 10} [get_ports CLKM]
create_clock -name CLKQ -period 10 -waveform {0 5}
create_clock -name CLKP \
  -period 5 -waveform {0 2.5} [get_ports CLKP]
```

A common base period of 20ns, as shown in Figure 8-33, is used when analyzing paths between the *CLKP* and *CLKM* clock domains.

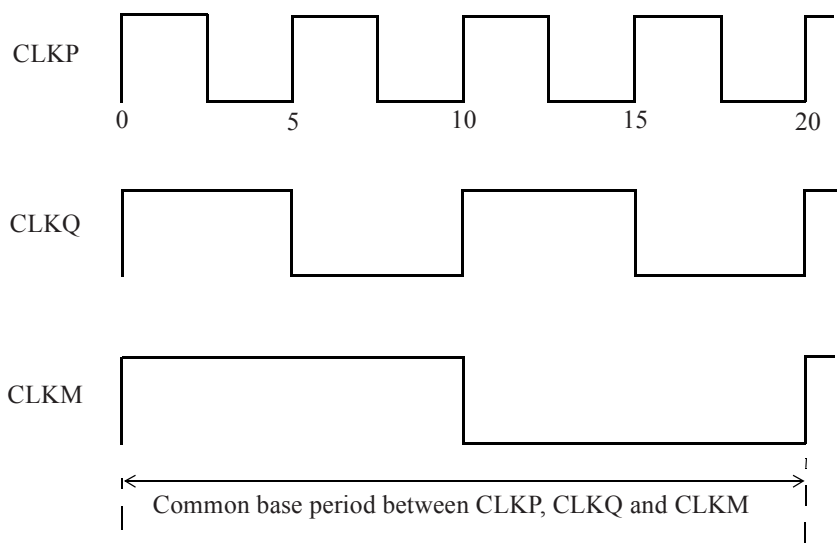


Figure 8-33 *Integer multiple clocks.*

Expanding clock 'CLKP' to base period of 20.00 (old period was 5.00, added 6 edges).

Expanding clock 'CLKQ' to base period of 20.00 (old period was 10.00, added 2 edges).

Here is a setup timing report for a path that goes from the faster clock to the slower clock.

Startpoint: UFF3 (rising edge-triggered flip-flop clocked by **CLKP**)
Endpoint: UFF1 (rising edge-triggered flip-flop clocked by **CLKM**)
Path Group: **CLKM**
Path Type: **max**

Point	Incr	Path

clock CLKP (rise edge)	15.00	15.00
clock source latency	0.00	15.00
CLKP (in)	0.00	15.00 r
UCKBUF4/C (CKB)	0.07	15.07 r
UFF3/CK (DFF)	0.00	15.07 r
UFF3/Q (DFF) <-	0.15	15.22 f
UNOR0/ZN (NR2)	0.05	15.27 r
UBUF4/Z (BUFF)	0.05	15.32 r
UFF1/D (DFF)	0.00	15.32 r
data arrival time		15.32
clock CLKM (rise edge)	20.00	20.00
clock source latency	0.00	20.00
CLKM (in)	0.00	20.00 r
UCKBUF0/C (CKB)	0.06	20.06 r
UCKBUF2/C (CKB)	0.07	20.12 r
UFF1/CK (DFF)	0.00	20.12 r
clock uncertainty	-0.30	19.82
library setup time	-0.04	19.78
data required time		19.78

data required time		19.78
data arrival time		-15.32

slack (MET)		4.46

Here is the corresponding hold path report.

Startpoint: UFF3 (rising edge-triggered flip-flop clocked by **CLKP**)

Endpoint: UFF1 (rising edge-triggered flip-flop clocked by **CLKM**)
Path Group: **CLKM**
Path Type: **min**

Point	Incr	Path

clock CLKP (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKP (in)	0.00	0.00 r
UCKBUF4/C (CKB)	0.07	0.07 r
UFF3/CK (DFF)	0.00	0.07 r
UFF3/Q (DFF) <-	0.16	0.22 r
UNOR0/ZN (NR2)	0.02	0.25 f
UBUF4/Z (BUFF)	0.06	0.30 f
UFF1/D (DFF)	0.00	0.30 f
data arrival time		0.30
clock CLKM (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKM (in)	0.00	0.00 r
UCKBUF0/C (CKB)	0.06	0.06 r
UCKBUF2/C (CKB)	0.07	0.12 r
UFF1/CK (DFF)	0.00	0.12 r
clock uncertainty	0.05	0.17
library hold time	0.01	0.19
data required time		0.19

data required time		0.19
data arrival time		-0.30

slack (MET)		0.12

8.10.2 Non-Integer Multiples

Consider the case when there is a data path between two clock domains whose frequencies are not multiples of each other. For example, the launch clock is divide-by-8 of a common clock and the capture clock is divide-by-5 of the common clock as shown in Figure 8-34. This section describes how the setup and hold checks are performed in such a situation.

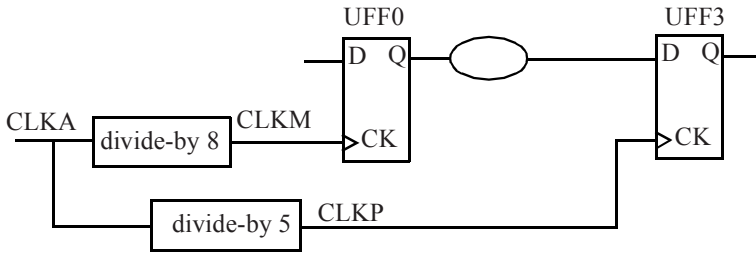


Figure 8-34 *Non-integer multiple clocks.*

Here are the clock definitions (waveforms are shown in Figure 8-35).

```
create_clock -name CLKM \
  -period 8 -waveform {0 4} [get_ports CLKM]
create_clock -name CLKQ -period 10 -waveform {0 5}
create_clock -name CLKP \
  -period 5 -waveform {0 2.5} [get_ports CLKP]
```

The timing analysis process computes a common period for the related clocks, and the clocks are then expanded to this base period. Note that the common period is found only for related clocks (that is, clocks that have timing paths between them). The common period for data paths between *CLKQ* and *CLKP* is expanded to a base period of 10ns only. The common period for data paths between *CLKM* and *CLKQ* is 40ns, and the common period for data paths between *CLKM* and *CLKP* is also 40ns.

Let us consider a data path from the *CLKM* clock domain to the *CLKP* clock domain. The common base period for timing analysis is 40ns.

```
Expanding clock 'CLKM' to base period of 40.00 (old period was
8.00, added 8 edges).
Expanding clock 'CLKP' to base period of 40.00 (old period was
5.00, added 14 edges).
```

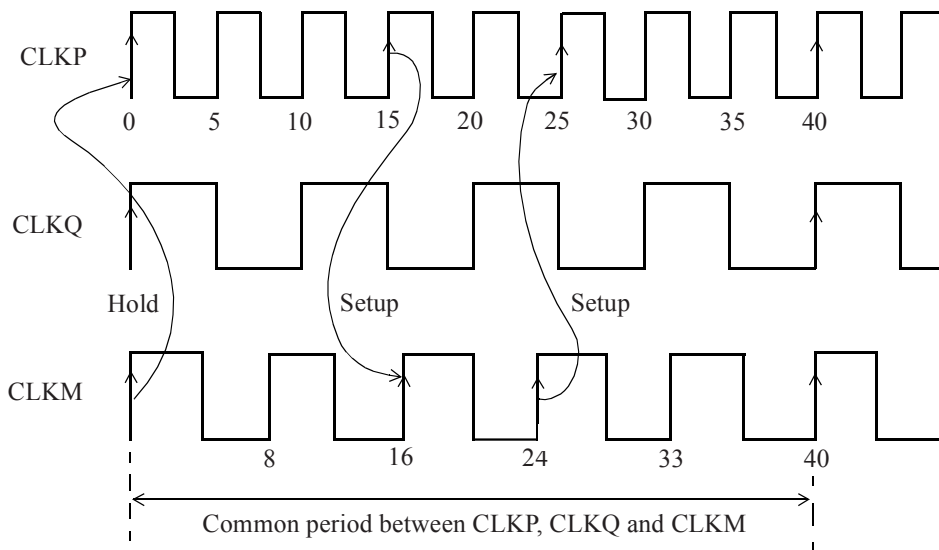


Figure 8-35 Setup and hold checks for non-integer multiple clocks.

The setup check occurs over the minimum time between the launch edge and the capture edge of the clock. In our example path from *CLKM* to *CLKP*, this would be a launch at 24ns of clock *CLKM* and a capture at 25ns of clock *CLKP*.

```
Startpoint: UFF0 (rising edge-triggered flip-flop clocked by CLKM)
Endpoint: UFF3 (rising edge-triggered flip-flop clocked by CLKP)
Path Group: CLKP
Path Type: max
```

Point	Incr	Path

clock CLKM (rise edge)	24.00	24.00
clock source latency	0.00	24.00
CLKM (in)	0.00	24.00 r

UCKBUF0/C (CKB)	0.06	24.06 r
UCKBUF1/C (CKB)	0.06	24.11 r
UFF0/CK (DFF)	0.00	24.11 r
UFF0/Q (DFF) <-	0.14	24.26 f
UNAND0/ZN (ND2)	0.03	24.29 r
UFF3/D (DFF)	0.00	24.29 r
data arrival time		24.29
clock CLKP (rise edge)	25.00	25.00
clock source latency	0.00	25.00
CLKP (in)	0.00	25.00 r
UCKBUF4/C (CKB)	0.07	25.07 r
UFF3/CK (DFF)	0.00	25.07 r
clock uncertainty	-0.30	24.77
library setup time	-0.04	24.72
data required time		24.72

data required time		24.72
data arrival time		-24.29

slack (MET)		0.44

Here is the hold timing path. The most restrictive hold path is from a launch at 0ns of *CLKM* to the capture edge of 0ns of *CLKP*.

Startpoint: UFF0 (rising edge-triggered flip-flop clocked by **CLKM**)
 Endpoint: UFF3 (rising edge-triggered flip-flop clocked by **CLKP**)
 Path Group: **CLKP**
 Path Type: **min**

Point	Incr	Path

clock CLKM (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKM (in)	0.00	0.00 r
UCKBUF0/C (CKB)	0.06	0.06 r
UCKBUF1/C (CKB)	0.06	0.11 r
UFF0/CK (DFF)	0.00	0.11 r
UFF0/Q (DFF) <-	0.14	0.26 r
UNAND0/ZN (ND2)	0.03	0.29 f

UFF3/D (DFF)	0.00	0.29 f
data arrival time		0.29
clock CLKP (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKP (in)	0.00	0.00 r
UCKBUF4/C (CKB)	0.07	0.07 r
UFF3/CK (DFF)	0.00	0.07 r
clock uncertainty	0.05	0.12
library hold time	0.02	0.13
data required time		0.13

data required time		0.13
data arrival time		-0.29

slack (MET)		0.16

Now we examine the setup path from the *CLKP* clock domain to the *CLKM* clock domain. In this case, the most restrictive setup path is from a launch edge at 15ns of clock *CLKP* to the capture edge at 16ns of clock *CLKM*.

Startpoint: UFF3 (rising edge-triggered flip-flop clocked by **CLKP**)
Endpoint: UFF1 (rising edge-triggered flip-flop clocked by **CLKM**)
Path Group: **CLKM**
Path Type: **max**

Point	Incr	Path

clock CLKP (rise edge)	15.00	15.00
clock source latency	0.00	15.00
CLKP (in)	0.00	15.00 r
UCKBUF4/C (CKB)	0.07	15.07 r
UFF3/CK (DFF)	0.00	15.07 r
UFF3/Q (DFF) <-	0.15	15.22 f
UNOR0/ZN (NR2)	0.05	15.27 r
UBUF4/Z (BUFF)	0.05	15.32 r
UFF1/D (DFF)	0.00	15.32 r
data arrival time		15.32
clock CLKM (rise edge)	16.00	16.00

clock source latency	0.00	16.00
CLKM (in)	0.00	16.00 r
UCKBUF0/C (CKB)	0.06	16.06 r
UCKBUF2/C (CKB)	0.07	16.12 r
UFF1/CK (DFF)	0.00	16.12 r
clock uncertainty	-0.30	15.82
library setup time	-0.04	15.78
data required time		15.78

data required time		15.78
data arrival time		-15.32

slack (MET)		0.46

Here is the hold path report, again the most restrictive one is the one at 0ns.

Startpoint: UFF3 (rising edge-triggered flip-flop clocked by **CLKP**)
 Endpoint: UFF1 (rising edge-triggered flip-flop clocked by **CLKM**)
 Path Group: **CLKM**
 Path Type: **min**

Point	Incr	Path

clock CLKP (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKP (in)	0.00	0.00 r
UCKBUF4/C (CKB)	0.07	0.07 r
UFF3/CK (DFF)	0.00	0.07 r
UFF3/Q (DFF) <-	0.16	0.22 r
UNOR0/ZN (NR2)	0.02	0.25 f
UBUF4/Z (BUFF)	0.06	0.30 f
UFF1/D (DFF)	0.00	0.30 f
data arrival time		0.30
clock CLKM (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CLKM (in)	0.00	0.00 r
UCKBUF0/C (CKB)	0.06	0.06 r
UCKBUF2/C (CKB)	0.07	0.12 r
UFF1/CK (DFF)	0.00	0.12 r
clock uncertainty	0.05	0.17

library hold time	0.01	0.19
data required time		0.19

data required time		0.19
data arrival time		-0.30

slack (MET)		0.12

8.10.3 Phase Shifted

Here is an example where two clocks are ninety degrees phase-shifted with respect to each other.

```
create_clock -period 2.0 -waveform {0 1.0} [get_ports CKM]
create_clock -period 2.0 -waveform {0.5 1.5} \
[get_ports CKM90]
```

Figure 8-36 shows an example with these clocks. The setup path timing report is as follows.

Startpoint: UFF0 (rising edge-triggered flip-flop clocked by **CKM**)
Endpoint: UFF3 (rising edge-triggered flip-flop clocked by **CKM90**)
Path Group: **CKM90**
Path Type: **max**

Point	Incr	Path

clock CKM (rise edge)	0.00	0.00
clock source latency	0.00	0.00
CKM (in)	0.00	0.00 r
UCKBUF0/C (CKB)	0.06	0.06 r
UCKBUF1/C (CKB)	0.06	0.11 r
UFF0/CK (DF)	0.00	0.11 r
UFF0/Q (DF) <-	0.14	0.26 f
UNAND0/ZN (ND2)	0.03	0.29 r
UFF3/D (DF)	0.00	0.29 r
data arrival time		0.29
clock CKM90(rise edge)	0.50	0.50
clock source latency	0.00	0.50

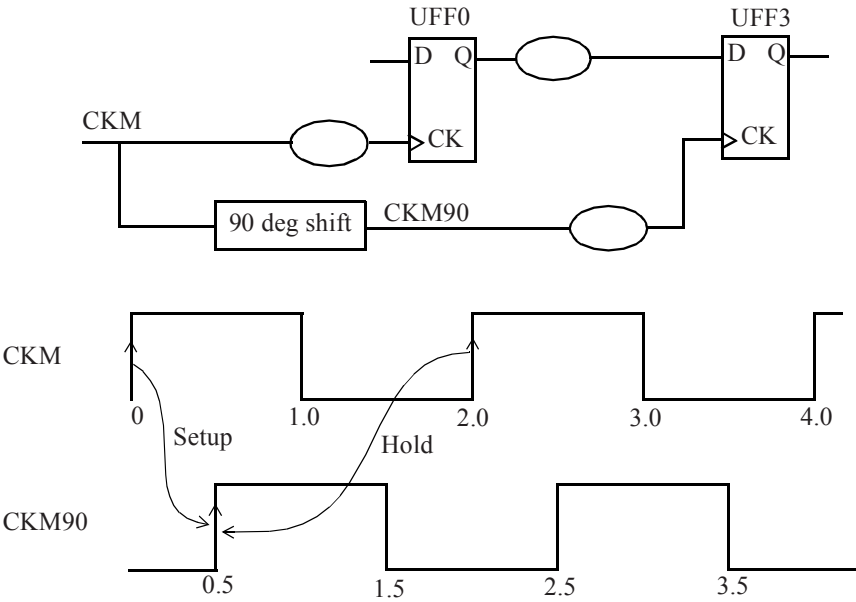


Figure 8-36 *Phase-shifted clocks.*

CKM90 (in)	0.00	0.50 r
UCKBUF4/C (CKB)	0.07	0.57 r
UFF3/CK (DF)	0.00	0.57 r
clock uncertainty	-0.30	0.27
library setup time	-0.04	0.22
data required time		0.22

data required time		0.22
data arrival time		-0.29

slack (VIOLATED)		-0.06

The first rising edge of *CKM90* at 0.5ns is the capture edge. The hold check is for one cycle before the setup capture edge. For the launch edge at 2ns,

the setup capture edge is at 2.5ns. Thus the hold check is at the previous capture edge which is at 0.5ns. The hold path timing report is as follows.

Startpoint: UFF0 (rising edge-triggered flip-flop clocked by **CKM**)
Endpoint: UFF3 (rising edge-triggered flip-flop clocked by **CKM90**)
Path Group: **CKM90**
Path Type: **min**

Point	Incr	Path

clock CKM (rise edge)	2.00	2.00
clock source latency	0.00	2.00
CKM (in)	0.00	2.00 r
UCKBUF0/C (CKB)	0.06	2.06 r
UCKBUF1/C (CKB)	0.06	2.11 r
UFF0/CK (DF)	0.00	2.11 r
UFF0/Q (DF) <-	0.14	2.26 r
UNAND0/ZN (ND2)	0.03	2.29 f
UFF3/D (DF)	0.00	2.29 f
data arrival time		2.29
clock CKM90(rise edge)	0.50	0.50
clock source latency	0.00	0.50
CLM90(in)	0.00	0.50 r
UCKBUF4/C (CKB)	0.07	0.57 r
UFF3/CK (DF)	0.00	0.57 r
clock uncertainty	0.05	0.62
library hold time	0.02	0.63
data required time		0.63

data required time		0.63
data arrival time		-2.29

slack (MET)		1.66

Other timing checks such as data to data checks and clock gating checks are described in Chapter 10.

